

THE UNIVERSITY OF DODOMA



COLLEGE OF INFORMATICS AND VIRTUAL EDUCATION DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CSE)

FINAL YEAR PROJECT PROPOSAL

ACADEMIC YEAR: **2025/2026**

TITLE: Threat Hunting Software for Detecting Credential-Based Lateral Movement APTs in Enterprise Networks

GROUP 3 MEMBERS:

#	NAME	REG. No	DEGREE	PHONE	SIGN
1	DANIEL T. LEMUNGE	T22-03-00362	Bsc. CSDFE	0763706818	
2	JOSTEN J. NYINGI	T21-03-01211	Bsc. CSDFE	0658185242	
3	IDDI B. HEMEDI	T22-03-05996	Bsc. CSDFE	0678506884	
4	MARATO CHACHA	T22-03-05970	Bsc. CSDFE	0744604912	
5	MARY M. MSOVELA	T22-03-00104	Bsc. CSDFE	0679335147	

NAME OF SUPERVISOR

SIGNATURE

Dr/Mr/Ms

.....

Table of Contents

CHAPTER ONE.....	1
1 INTRODUCTION.....	1
1.1 Project Overview.....	1
1.2 Problem Statement.....	1
1.3 Objectives.....	2
1.3.1 Main Objective.....	2
1.3.2 Specific Objectives.....	2
1.4 Project Significance.....	2
1.5 Project Scope.....	2
CHAPTER TWO.....	3
2 LITERATURE REVIEW.....	3
2.1 Introduction.....	3
2.2 Definitions of Key Terms.....	3
2.3 Theoretical Literature/Framework.....	4
2.3.1 The Problem of Lateral Movement in Enterprise Networks.....	4
2.3.2 The Cyber Kill Chain and MITRE ATT&CK Framework.....	5
2.3.3 Machine Learning for Threat Detection.....	5
2.3.4 Federated Learning for Privacy-Preserving Security Analytics.....	6
2.3.5 Threat Intelligence Integration and Correlation.....	6
2.3.6 Graph-Based Attack Path Analysis and Visualization.....	6
2.4 Related (Similar) Work.....	7
2.4.1 Commercial APT Detection Platforms.....	7
2.4.2 Open-Source Security Projects.....	7
2.4.3 Academic Research Projects.....	8
2.5 Innovation/Research Gap.....	9
CHAPTER THREE.....	10
3 METHODOLOGY.....	10
3.1 Introduction.....	10
3.2 Research Approach.....	10
3.2.1 DSR Alignment.....	10
3.2.2 Justification within DSR.....	11
3.3 Research Method.....	11
3.3.1 DSR Process Application.....	11
3.4 Study Area / Location.....	11
3.4.1 Contextual Justification.....	11
3.4.2 Ethical Considerations.....	12
3.5 Implementation Steps (DSR Design & Development Phase).....	12
3.5.1 Modular Threat Hunting Platform Architecture Design.....	12
3.5.2 XGBoost machine learning classifier Implementation.....	12
3.5.3 Supporting Platform Subsystems Implementation.....	13
3.5.4 Detection Performance Evaluation against Defined Thresholds.....	14
3.6 Data Collection / Requirements Gathering.....	15
3.6.1 Data Collection Techniques/Methods.....	15
3.6.2 Data Collection Tools.....	16
3.7 System Requirements.....	17
3.7.1 Hardware Requirements.....	17
3.7.2 Software Requirements.....	17
CHAPTER FOUR.....	18

4	PROJECT TIMELINE AND BUDGET.....	18
4.1	Project Timeline.....	18
4.1.1	Gantt Chart - Project Timeline.....	18
4.2	Project Budget.....	19
4.2.1	Detailed Budget Breakdown.....	19
	CHAPTER FIVE.....	20
5	PROJECT DESIGN.....	20
5.1	Introduction.....	20
5.2	Functional and Non-Functional Requirements.....	20
5.2.1	Functional Requirements.....	20
5.2.2	Non-functional Requirements.....	21
5.2.3	Security Requirements.....	21
5.3	System Design.....	23
5.4	Achievements so far.....	37
5.4.1	Objective 1: Design.....	37
5.4.2	Objective 2: Detection Engine Implementation.....	37
5.4.3	Objective 3: Platform Subsystems Implementation.....	38
5.5	Current Progress Status.....	39
	REFERENCES.....	40

List of Figures

Figure 1: Project timeline.....	18
Figure 2: Threat Hunting Platform Architecture Design.....	23
Figure 3: Threat Hunting Platform UseCase Diagram.....	24
Figure 4: Threat Hunting Platform Usecase Diagram for Federated Learning Subsystem.....	25
Figure 5: Threat Hunting Platform Sequence Diagram (API auth & RBAC).....	26
Figure 6: Threat Hunting Platform Sequence Diagram (threat detection pipeline).....	26
Figure 7: Threat Hunting Platform Sequence Diagram (federated learning).....	27
Figure 8: Threat Hunting Platform Activity Diagram (detection engine).....	28
Figure 9: Threat Hunting Platform Activity Diagram (alert propagation).....	29
Figure 10: Threat Hunting Platform Context Diagram.....	30
Figure 11: Threat Hunting Platform DFD level 1.....	31
Figure 12: Threat Hunting Platform DFD level 2 (threat detection).....	32
Figure 13: Threat Hunting Platform DFD level 2 (data ingestion).....	32
Figure 14: Threat Hunting Platform DFD level 2 (feature engineering).....	33
Figure 15: Threat Hunting Platform DFD level 2 (intelligence enrichment).....	33
Figure 16: Threat Hunting Platform DFD level 2 (alert management).....	34
Figure 17: Threat Hunting Platform DFD level 2 (attack graph visualization).....	34
Figure 18: Threat Hunting Platform DFD level 2 (federated learning).....	35
Figure 19: Threat Hunting Platform state diagram (detection engine).....	35
Figure 20: Threat Hunting Platform state diagram (federated learning client).....	36
Figure 21: Threat Hunting Platform state diagram (alert lifecycle).....	36

List of Tables

Table 1: Data Collection Tools.....	16
Table 2: Central Server Hardware Requirements.....	17
Table 3: Endpoints Hardware Requirements.....	17
Table 4: Central Server Software Requirements.....	17
Table 5: Endpoints Software Requirements.....	17
Table 6: Project Budget Breakdown.....	19
Table 7: Threat Hunting Platform Functional Requirements.....	20
Table 8: Threat Hunting Platform Non-functional Requirements.....	21
Table 9: Threat Hunting Platform Security Requirements.....	21
Table 10: Threat Hunting Platform objective 1 progress.....	37
Table 11: Threat Hunting Platform objective 2 progress.....	37
Table 12: Threat Hunting Platform objective 3 progress.....	38

ABBREVIATIONS

CBLM	: Credential-Based Lateral Movement
APT	: Advanced Persistent Threat
IDS	: Intrusion Detection System
IPS	: Intrusion Prevention System
AI	: Artificial Intelligence
ML	: Machine Learning
XGBoost	: Extreme Gradient Boost
SIEM	: Security Information and Event Management
MISP	: Malware Information Sharing Platform
IoC	: Indicators of Compromise
MTTD	: Mean Time To Detect
MTTR	: Mean Time To Respond
PtH	: Pass-the-Hash
PtT	: Pass-the-Ticket
XDR	: Extended Detection and Responce
TTP	: Tactics, Techniques and Procedure
CNN	: Convolution Neural Network
LSTM	: Long Short-Term Memory
DSR	: Design Science Research
SecureUML	: Secure Unified Modeling Language
JWT	: JSON Web Token
REST API	: Representational State Transfer Application Programming Interface
ROC-AUC	: Receiver Operating Characteristic - Area Under Curve
FPR	: False Positive Rate
HMAC	: Hash-based Message Authentication Code
FL	: Federated Learning
SHAP	: SHapley Additive exPlanations
HDD	: Hard Disk Drive
DFD	: Data Flow Diagram
HTTP	: Hyper Text Transfer Protocol
HTTPS	: Secure Hyper Text Transfer Protocol
RBAC	: Role-Based Access Control
FR	: Functional Requirements
NFR	: Non-Functional Requirements

CHAPTER ONE

1 INTRODUCTION

1.1 Project Overview

Traditional Network Security solutions such as signature-based antivirus, firewalls, and conventional Intrusion Detection Systems (IDS) are ineffective against Advanced Persistent Threats (APTs) due to their reliance on known attack patterns and rule-based detection mechanisms.

APTs in enterprise networks typically manifest in several overlapping forms based on their objectives and tactics.

Espionage-focused APTs aim to silently exfiltrate sensitive data over long periods.

Financially motivated APTs targets payment infrastructure for direct monetary gain.

Destructive APTs are designed to disrupt operations through data wiping or sabotage.

Supply chain APTs infiltrate trusted software to gain indirect access to enterprise networks.

Zero-Day Exploit APTs leverages previously unknown vulnerabilities on installed software

Credential-based lateral movement APTs focus on stealing and reusing authentication data to move stealthily across systems, escalating privileges and maintaining persistence.

Across all types, APTs are characterized by their stealth, persistence, and use of advanced techniques such as zero-day exploits, social engineering, and living-off-the-land tools to evade detection while maintaining long-term access.

1.2 Problem Statement

Cyber attackers have become more advanced and persistent, using set of repetitive, yet sophisticated attacks, known as Advanced Persistent Threats (APTs). They are difficult to detect as they hide within normal network traffic and can remain undetected for long periods.

Most existing security systems such as firewalls and antivirus tools are not strong enough to detect or stop these advanced attacks. They often rely on known attack patterns and fail to recognize new, unknown threats.

There is a strong need for a modern, intelligent threat hunting platform that can automatically detect, analyze, and trace these advanced threats using Artificial Intelligence (AI) and Machine Learning (ML) techniques.

This project will solely focus on Credential-Based and Lateral Movements detection from attackers as they infiltrate an Enterprise network.

1.3 Objectives

1.3.1 Main Objective

The main objective of this project is to develop a Threat Hunting Software tool that can detect Credential-Based and Lateral Movement (CBLM) APTs within an enterprise network.

1.3.2 Specific Objectives

- i. To Analyze existing Network Security solutions on Enterprise Networks.
- ii. To Design a modular threat hunting platform architecture with integrated security controls for detecting CBLM APTs.
- iii. To Implement XGBoost machine learning classifier for CBLM detection integrated bidirectionally with Wazuh SIEM for telemetry ingestion and alert publishing.
- iv. To Implement the supporting platform subsystems that transform the detection engine into a deployable threat hunting platform for detecting CBLM APTs.
- v. To Evaluate CBLM detection performance against defined thresholds and assess software usability through demonstration sessions.

1.4 Project Significance

The study is significant in Improving Enterprise Security posture and incident response, Reducing Financial and Data Loss on the organizations. It will further enhance Cyber Threat Intelligence sharing between organizations, securing wider range of organizations. The project will inturn Provide a Learning Platform for cyber security analysts to tweak performance/security priorities as needed, promoting Automation in Cyber Defense.

1.5 Project Scope

The scope of the project is limited to Credential-based lateral movement APT detection in a simulated Enterprise Network; Specifically detecting abnormal authentication and credential reuse patterns, multiple use of same credentials, privilege escalations, credential harvesting, remote logon from non-admin user, session impersonations, log clearing.

The project will be interoperable with existing well-known open source enterprise tool, wazuh. Making it easy for security experts to familiarize and harness the capability of machine learning tools on a rule-based platform.

CHAPTER TWO

2 LITERATURE REVIEW

2.1 Introduction

This chapter presents a review of existing literature relevant to the development of a Threat Hunting Platform for Detecting Credential-Based Lateral Movement Advanced Persistent Threats (APTs) in Enterprise Networks. The chapter examines theoretical foundations, technical frameworks, and related research that informed the design and implementation of the proposed system. It covers key concepts including lateral movement detection, machine learning for cybersecurity, federated learning architectures, threat intelligence integration, and attack path visualization; identifying existing gaps in current solutions and establishing the rationale for our proposed research.

2.2 Definitions of Key Terms

Advanced Persistent Threat (APT): A sophisticated, targeted cyberattack conducted by well-resourced adversaries (nation-state actors or organized cybercrime groups) who employ multiple attack vectors and techniques to establish and maintain long-term unauthorized access to target networks.

Credential-Based Attack: An unauthorized use of legitimate user credentials to access systems and resources. Common techniques include credential dumping, Pass-the-Hash (PtH), Pass-the-Ticket, and token impersonation.

Lateral Movement: Techniques that adversaries use after compromising an initial system to move through a network in search of key data and assets, privilege escalation or establishment of persistence in critical systems.

Machine Learning (ML): A subset of artificial intelligence that enables computer systems to automatically learn patterns and make decisions from data without explicit programming.

XGBoost (Extreme Gradient Boosting): An optimized distributed gradient boosting machine learning library that implements unsupervised learning through decision tree algorithms.

Federated Learning: A distributed machine learning approach where multiple participants (organizations, devices, or nodes) collaboratively train a shared model while keeping their training data localized and private (maintaining data sovereignty and privacy).

Security Information and Event Management (SIEM): A security software solution that aggregates, normalizes, correlates, and analyzes log data from diverse sources across an organization's IT infrastructure to provide centralized visibility, compliance reporting, and security alerting.

Wazuh: An open-source security monitoring platform that provides unified XDR (Extended Detection and Response) and SIEM capabilities for endpoint security, vulnerability detection, compliance monitoring, and threat intelligence integration.

MITRE ATT&CK Framework: A globally accessible knowledge base of adversary tactics, techniques, and procedures (TTPs) based on real-world observations of cyberattacks.

MISP (Malware Information Sharing Platform): An open-source threat intelligence platform designed to improve sharing of structured threat information (indicators of compromise IoCs, threat actor information) between organizations.

Behavioral Analysis/Anomaly Detection: Security monitoring approach that establishes baselines of normal user, system, and network behavior, then identifies deviations from these baselines as potential security incidents.

Indicators of Compromise (IoCs): Forensic artifacts or observable evidence that suggest a system has been compromised or attacked.

Mean Time to Detect (MTTD) / Mean Time to Respond (MTTR): Key cybersecurity metrics measuring incident response effectiveness. MTTD measures the average time between initial compromise and detection of the security incident. MTTR measures the average time from detection to complete incident resolution and system recovery.

Feature Engineering: The process of transforming raw data into meaningful numerical features that machine learning algorithms can effectively process.

False Positive / False Negative Rate: Critical performance metrics for security detection systems. *False positives* occur when benign activity is incorrectly classified as malicious, creating alert fatigue. *False negatives* occur when actual attacks are missed by the detection system, leaving the organization vulnerable.

Threat Intelligence: Evidence-based knowledge about existing or emerging threats to assets, including context, mechanisms, indicators, implications, and actionable advice.

Sysmon (System Monitor): Sysmon is a Windows system service and device driver that monitors and logs system activity to the Windows event log.

Attack Graph/Path Visualization: A visualization is a technique for representing potential attack paths through a network as a directed graph.

2.3 Theoretical Literature/Framework

The theoretical foundation for this project draws from multiple domains including machine learning theory, network security principles, collaborative learning frameworks, and behavioral analytics. Understanding these theoretical underpinnings is essential to justify technical design decisions and establish the scientific basis for the proposed approach.

2.3.1 The Problem of Lateral Movement in Enterprise Networks

As noted by Smiliotopoulos et al. (2024) in their systematic survey on lateral movement detection, this tactic is a principal component in the arsenal of APT groups and sophisticated threat actors. Once attackers establish an initial foothold within a network, they progressively move through the infiltrated environment, gaining elevated privileges and seeking valuable data for exfiltration.

The challenge of detecting lateral movement stems from several factors:

First, attackers increasingly leverage legitimate tools and valid credentials to move through networks, a technique known as "living off the land" that produces minimal anomalous signatures.

Second, the volume and velocity of security telemetry in enterprise environments make manual analysis impractical.

Third, traditional rule-based detection systems struggle to identify novel attack patterns that deviate from known signatures (HawkEye, 2024).

This necessitates behavioral approaches that can identify anomalous authentication patterns without relying on explicit indicators of compromise.

2.3.2 The Cyber Kill Chain and MITRE ATT&CK Framework

The MITRE ATT&CK framework categorizes lateral movement under tactic TA0008, encompassing techniques such as Remote Services (T1021), Internal Spearphishing (T1534), and Lateral Tool Transfer (T1570). The framework provides a standardized vocabulary for describing adversary behavior, facilitating seamless integration of threat intelligence feeds and enabling correlation of observed activities with known attack patterns.

The ATT&CK framework has become instrumental in threat-informed defense strategies. Organizations use it to map their detection capabilities against known adversary techniques, identify gaps in coverage, and prioritize security investments.

2.3.3 Machine Learning for Threat Detection

Machine Learning techniques offer the ability to identify complex patterns, detect previously unknown threats, and adapt to evolving attack strategies. In the context of lateral movement detection, ML approaches can analyze behavioral patterns across authentication events, process executions, and network communications to identify anomalous activities.

Gouveia and Correia (2023) conducted extensive research on network intrusion detection using XGBoost, demonstrating that gradient boosting algorithms achieve superior performance in classifying security events. Their work established that XGBoost outperforms other classifiers in terms of accuracy, precision, and recall, while maintaining computational efficiency suitable for real-time detection scenarios.

A comparative study by Bhardwaj et al. (2025) evaluated machine learning and deep learning models for intrusion detection, finding that Random Forest achieved the highest F1-score of 93.57% on industrial control system datasets, while XGBoost attained a near-perfect F1-score of 99.97% on software-defined networking datasets. These results demonstrate that ensemble methods, particularly gradient boosting approaches, consistently outperform deep learning models for binary and multiclass classification in intrusion detection contexts.

Research on malware classification by Taheri and Shameli-Sendi (2020) demonstrated that XGBoost can be optimized for efficiency through feature reduction and hyperparameter tuning, achieving accuracy of 98.5% with reduced computational resources.

2.3.4 Federated Learning for Privacy-Preserving Security Analytics

According to recent research published in Applied Sciences (2025), federated learning frameworks for intrusion detection enable decentralized model training while preserving data locality and minimizing communication overhead. The proposed architecture incorporates lightweight privacy-preserving techniques including gradient clipping, differential privacy, and encrypted model aggregation to ensure secure collaboration across heterogeneous clients.

Sharma and Yadav (2025) demonstrated that federated learning offers a safer alternative for intrusion detection systems, allowing multiple clients to collaborate on model development without sharing raw data.

The Privacy-Enhanced IoT Defence System (PEIoT-DS) presented by researchers in Discover Internet of Things (2025) utilized federated learning to create a comprehensive intrusion detection model where IoT devices contribute only model updates rather than raw data.

Research on federated learning in industrial cyber-physical systems by Zhang et al. (2023) introduced DeepFed, a federated deep learning scheme combining CNN and GRU techniques. The findings demonstrated effectiveness using real-world industrial datasets, highlighting the potential for federated approaches to bolster cybersecurity in complex operational environments.

2.3.5 Threat Intelligence Integration and Correlation

MISP, the Malware Information Sharing Platform, is designed by and for incident analysts to support day-to-day operations in sharing structured threat information (CIRCL, 2024). The platform provides functionalities to support information exchange and consumption by Network Intrusion Detection Systems, Security Information and Event Management systems, and log analysis tools.

MISP integrates intelligence vocabularies including MITRE ATT&CK, enabling organizations to tag events with standardized technique identifiers. According to Anomali (2024), threat intelligence becomes actionable when integrated with security platforms through common frameworks like ATT&CK.

Security Information and Event Management platforms can provide richer context for alerts when they correlate detected activities with known ATT&CK techniques. For example, detection of suspicious PowerShell script execution can be mapped to technique T1059.001, enabling automated response workflows and contextual alerting.

2.3.6 Graph-Based Attack Path Analysis and Visualization

Research by Neo4j (2025) demonstrates that graph databases and visualization tools enable security teams to preempt attack paths by analyzing how assets are connected and identifying vulnerable paths. By picturing the system as a graph, defenders can conceptualize the same attack paths that adversaries would exploit, enabling proactive security measures.

Noel et al. (2016) developed CyGraph, a graph-based analytics and visualization system for cybersecurity that represents network infrastructure, vulnerabilities, and potential attack paths as a property graph. Using Neo4j and the Cypher query language, CyGraph enables traversal of attack graphs to identify paths of exploitable vulnerabilities from external entry points to critical assets.

2.4 Related (Similar) Work

This section reviews existing academic research, commercial products, and open-source projects related to APT detection, machine learning-based threat hunting, and collaborative defense mechanisms. Understanding prior work enables us to position our contribution and avoid duplicating existing solutions.

2.4.1 Commercial APT Detection Platforms

- i. **Darktrace Enterprise Immune System:** Darktrace employs unsupervised machine learning and self-learning AI to detect threats by modeling normal behavior patterns and identifying deviations. The system uses clustering algorithms and Bayesian networks to detect novel threats without relying on signatures (Darktrace, 2023). While highly effective, Darktrace's commercial licensing (\$100,000+ annually) makes it inaccessible to most organizations in developing regions.
- ii. **Vectra Cognito Platform:** Vectra uses supervised and unsupervised ML to detect attacker behaviors across network traffic and cloud workloads, with particular focus on lateral movement and data exfiltration phases of APT campaigns (Vectra AI, 2023). The platform provides attack path visualization similar to our proposed component. However, Vectra is proprietary software with opaque ML models, limiting customization and academic research opportunities.
- iii. **CrowdStrike Falcon:** Falcon's Threat Graph correlates trillions of security events to identify APT campaigns using ML-based behavioral analytics and threat intelligence (CrowdStrike, 2024). While effective, Falcon primarily focuses on endpoint detection rather than network-wide correlation and does not offer federated learning capabilities for cross-organization collaboration while preserving privacy.

Key Limitations of Commercial Solutions: All commercial platforms share common limitations:

- High cost barriers (\$50,000-\$500,000+ annually)
- Proprietary algorithms limiting transparency and customization
- Cloud-dependent architectures raising data sovereignty concerns
- No federated learning for collaborative defense while maintaining privacy

2.4.2 Open-Source Security Projects

- i. **Wazuh SIEM:** Wazuh provides open-source security monitoring with agent-based log collection, rule-based correlation, vulnerability detection, and compliance checking (Wazuh, 2024). While Wazuh excels at log aggregation and basic threat detection, it lacks:
 - Sophisticated ML-based behavioral analysis
 - APT-specific detection models
 - Advanced attack path visualization

- Automated MITRE ATT&CK mapping. Our project extends Wazuh with these missing capabilities.
- ii. **Security Onion:** Security Onion integrates multiple open-source security tools (Zeek, Suricata, Elasticsearch, Kibana) into a complete network security monitoring platform (Security Onion Solutions, 2024). However, detection remains primarily rule-based and signature-driven.

Gap in Open-Source Solutions: Current open-source security lack: (1) Pre-trained ML models specifically optimized for APT detection, (2) Federated learning frameworks for collaborative defense, (3) User-friendly attack path visualization. Our project synthesizes these components into a cohesive solution.

Research Gap: Academic research typically lacks: (1) Integration with operational security infrastructure (SIEM), (2) Federated learning for multi-organization collaboration, (3) Comprehensive threat intelligence enrichment,

2.4.3 Academic Research Projects

- "Deep Learning for APT Detection in Network Traffic" (Liu et al., 2019):** This IEEE research applied Long Short-Term Memory (LSTM) networks to network packet data for APT detection, achieving 89% accuracy. However, the study:
 - Used only network-level features, missing endpoint and authentication signals
 - Did not integrate with operational SIEM systems
 - Need computational resources for deep learning, making real-time deployment challenging.
- "XGBoost-Based Intrusion Detection System" (Dhaliwal et al., 2018):** This ACM paper demonstrated XGBoost's effectiveness for network intrusion detection on the NSL-KDD dataset, achieving 98.5% accuracy. However, the research focused on general intrusion detection rather than APT-specific behaviors, did not incorporate threat intelligence enrichment, and remained a theoretical model without operational SIEM integration.
- "Federated Learning for Cybersecurity: A Survey" (Nguyen et al., 2021):** This comprehensive survey reviewed federated learning applications in security, including intrusion detection and malware classification. The authors identified privacy preservation and collaborative defense as key benefits but noted *limited real-world implementations in operational security infrastructure*. Our project addresses this implementation gap by demonstrating federated learning integrated with Wazuh SIEM.
- "Behavioral Analytics for APT Detection Using Graph Analysis" (Milajerdi et al., 2019):** This USENIX Security paper proposed Holmes, a system that constructs attack graphs from system logs to identify APT campaigns. Holmes achieved high detection rates with low false positives by correlating events into attack scenarios.

However, the research:

- Required custom log parsing infrastructure rather than existing SIEM platforms
- Did not incorporate ML-based behavioral classification
- Lacked threat intelligence integration.

2.5 Innovation/Research Gap

Despite significant advances in lateral movement detection and threat hunting technologies, the literature review reveals several gaps and opportunities for innovation that the proposed system addresses:

Gap: Federated Learning for APT Detection with Privacy Preservation

Existing Situation: Current APT detection approaches require either: (1) Training ML models on single-organization data, limiting exposure to diverse attack patterns, or (2) Centralizing security logs from multiple organizations, creating privacy and sovereignty concerns. No existing open-source solution implements federated learning specifically for APT detection.

Our Innovation: We implement Flower framework to enable collaborative XGBoost model training across computer nodes. Computer nodes contribute to model improvement by sharing model parameters rather than raw security logs, maintaining data privacy while benefiting from collective threat intelligence. This addresses the fundamental limitation that APT tactics vary by target industry and geography.

Contribution: First open-source implementation of federated learning specifically optimized for APT detection that maintains compliance with data protection regulations while enabling cross-organization collaborative defense.

CHAPTER THREE

3 METHODOLOGY

3.1 Introduction

In this chapter, a formalized research approach based on **Design Science Research Methodology (DSR)** is presented. DSR provides a systematic, repeatable, and scientifically rigorous framework for solving real-world problems through the design, development, and evaluation of innovative artifacts.

This chapter details the methodology used in designing, developing, implementing, and evaluating the credential-based lateral movement Threat Hunting Platform for APT detection.

3.2 Research Approach

This study adopts a **Design Science Research (DSR) approach**, integrating both **quantitative and qualitative evaluation methods** within the DSR evaluation phase to ensure comprehensive assessment of the developed artifact.

3.2.1 DSR Alignment

The research approach is structured around the DSR phases; Problem Identification and Motivation, Define Objectives of a Solution, Design and Development, Demonstration, Evaluation, Communication

Quantitative Components (DSR Evaluation Phase)

The quantitative evaluation focuses on measuring the **performance** of the designed artifact. Quantitative data will be collected through Automated system logging, Model evaluation on test datasets, Benchmark testing using synthetic APT scenarios, Runtime performance profiling.

Statistical analysis will determine whether the artifact meets defined thresholds; $\geq 85\%$ precision, $\geq 90\%$ recall, $< 5\%$ false positive rate, < 2 minutes detection latency.

Qualitative Components (DSR Evaluation Phase)

Qualitative evaluation assesses the **practical utility and usability** of the artifact. Data collection methods will include Structured interviews with IT security staff, Observation during system usage, Think-aloud protocols, Open-ended questionnaires

3.2.2 Justification within DSR

Within DSR, evaluation must demonstrate both Rigor (quantitative performance validation) and Relevance (real-world usability and effectiveness). Thus, combining quantitative and qualitative methods ensures that the artifact is both **technically effective and operationally viable**.

3.3 Research Method

This project follows the **Design Science Research process model**, supported by **iterative and Agile-inspired development cycles** within the third phase (design and development phase).

3.3.1 DSR Process Application

1. **Problem Identification:** Understanding limitations in detecting credential-based lateral movement
2. **Objectives Definition:** Establishing performance and functional goals
3. **Design & Development:** Building the threat hunting platform (Section 3.5)
4. **Demonstration:** Deploying and testing the system in controlled environments
5. **Evaluation:** Measuring performance and usability
6. **Communication:** Documenting and presenting findings

Agile within DSR

Agile practices are embedded specifically in the **Design and Development phase** to support Iterative refinement of machine learning models, Continuous integration of system components and Frequent feedback from stakeholders

Justification

Traditional waterfall approaches assume stable requirements, which is unsuitable for cybersecurity and machine learning systems. DSR combined with Agile allows Continuous experimentation, Adaptive system improvement and Iterative validation of artifact performance

3.4 Study Area / Location

This study will be conducted at the **University of Dodoma (UDOM)**, utilizing sample computers from students volunteered to participate on this study. This will provide a necessary environment for data collection and detection simulation. Further more, we will use cloud service to simulate federated server dataset distribution across multiple computers for CBLM APT detection.

3.4.1 Contextual Justification

Data Availability: Access to realistic security logs (with anonymization)

Stakeholder Access: IT staff provide domain expertise

Federated Learning Nodes: Student computers as real nodes, Simulated financial and government nodes using synthetic data

3.4.2 Ethical Considerations

Study demands Informed consent from participants, Data anonymization, No active attacks on production systems and Use of controlled environments for testing.

3.5 Implementation Steps (DSR Design & Development Phase)

This section represents the **core artifact creation phase in DSR**, where the system is designed and developed iteratively. Each step contributes to the construction of the final artifact:

3.5.1 Modular Threat Hunting Platform Architecture Design

DSR Role: Platform Design

Inputs: MITRE ATT&CK Design Framework, SecureUML specification, functional requirements gathered from academic literature.

Method and Rationale: Design follows a contract-driven decoupled modular patterns, Docker orchestration with 3 isolated networks (agent-net, internal-net, fl-net). this enforces physical separation on concerns.

Tools: Python 3.11, Docker + Docker Compose, PyYAML, PyJWT, YAML configuration files (platform.yaml, wazuh.yaml, detectors.yaml, federated.yaml, security.yaml).

Expected Output: platform architecture specification

Validation Metrics: All modules independently importable with no circular dependencies, Securely communicating, Audit chain verification

3.5.2 XGBoost machine learning classifier Implementation

DSR Role: Platform Implementation

Inputs: Mordor Datasets, Wazuh SIEM REST API, MITRE ATT&CK technique databases.

Method and Rationale: CBLM patterns are chosen because generic network features fail to capture the nuances of credential abuse. Research demonstrates XGBoost achieving >98% accuracy on intrusion detection while maintaining efficiency.

Tools: XGBoost 1.7+, scikit-learn 1.3+, SHAP 0.42+, pandas + NumPy, Flower, FastAPI, Wazuh 4.7, httpx, Sysmon v15, NetworkX 3.1+, pyvis 0.3+, SQLite

Expected Output: trained XGBoost model file, feature engineering pipeline, Wazuh API connector, alert management, attack graph visualization

Validation Metrics: precision $\geq 85\%$, recall $\geq 90\%$, F1 $\geq 87\%$, ROC-AUC ≥ 0.90 , FPR $< 5\%$, Detection latency < 2 minutes at 95th percentile

3.5.3 Supporting Platform Subsystems Implementation

DSR Role: Platform Implementation

Inputs: Detection outputs from Objective 2, partitioned Mordor dataset, MITRE ATT&CK technique database, platform architecture design from Objective 1.

Method and Rationale: The detection engine from Objective 2 produces raw Detection objects, but a production threat hunting platform requires additional subsystems to make those detections operationally useful. This objective implements six subsystem that wrap around the engine:

(1) Federated Learning: FL operates at the organization level, not the laptop level, because a single laptop generates too few events for meaningful XGBoost training, lateral movement detection requires cross-endpoint context, and the real privacy boundary is inter-organizational.

(2) Threat Intelligence Enrichment: Detections are enriched with MITRE ATT&CK technique descriptions, tactic classifications, and recommended response actions. This transforms a raw "confidence 0.87 detection" into an actionable alert that tells an analyst exactly what attack technique was observed and what to do about it.

(3) Alert Management: Alerts are scored, deduplicated (preventing alert fatigue from repeated detections of the same ongoing attack), persisted to SQLite (surviving platform restarts), and published back to Wazuh Dashboard so they appear alongside native Wazuh alerts.

(4) Attack Path Visualization: NetworkX builds directed attack graphs from correlated detections (hosts as nodes, lateral movement and exfiltration as edges), and pyvis renders them as interactive HTML pages.

(5) Security Controls: JWT + API Key authentication with RBAC (4 roles, 11 permissions) enforced on every FastAPI endpoint, HMAC-SHA256 model integrity verification on every model load, hash-chained audit trail logging every security-relevant action.

(6) Platform Infrastructure: Async event bus decoupling all modules, structured JSON logging with correlation IDs for end-to-end tracing, health check protocol for monitoring, Docker Compose deployment with container security hardening.

Tools: Flower (flwr) 1.7+, NumPy, NetworkX 3.1+, pyvis 0.3+, SQLite, PyJWT, FastAPI, Docker + Docker Compose, Python logging + json.

Expected Output: a working threat detection platform that is deployable using Docker Compose with security-hardened containers.

Validation Metrics: Federated global model accuracy exceeds best individual client's local model accuracy by $\geq 3\%$. FL converges within 10 rounds. Differential privacy model ($\epsilon=1.0$) retains $\geq 95\%$. Trust manager correctly rejects corrupted contributions. Detection latency (event $\rightarrow$ alert): < 2 minutes at 95th percentile.

3.5.4 Detection Performance Evaluation against Defined Thresholds

DSR Role: Platform Demonstration and Evaluation

Inputs: Trained XGBoost models from Objective 2, federated global model from Objective 3, heldout test datasets (20% of Mordor)

Method and Rationale: Evaluation combines quantitative model performance assessment with qualitative usability testing.

Tools: scikit-learn, SHAP, Atomic Red Team, matplotlib + seaborn, platform health endpoint, Docker stats (container CPU/memory profiling), PyMISP, attacki, STIX2

Expected Output: Model performance report with precision/recall/F1/ROC-AUC for lateral movement, confusion matrices TP/FP/TN/FN distributions.

Validation Metrics: Lateral movement precision $\geq 85\%$, recall $\geq 90\%$, F1 $\geq 87\%$, ROC-AUC ≥ 0.93 , FPR $< 5\%$. Detection latency — < 2 minutes at 95th percentile.

3.6 Data Collection / Requirements Gathering

3.6.1 Data Collection Techniques/Methods

With reference to DSR, data collection will be done in multiple phases:

Problem Identification (requirements gathering): Requirements data was collected from Literature review through dissemination of existing works (IEEE Xplore, ACM Digital Library), commercial product analysis (Darktrace, CrowdStrike, Vectra) and existence of gaps in existing security tools.

Design (training data acquisition): Training data will be collected from students computers volunteered to participate in this study. We will also use publicly available datasets, Specifically, the Mordor Datasets, as it is enriched on CBLM APTs (~500K pre-recorded APT attack events with MITRE ATT&CK labels).

Demonstration (scenario simulation): Data collection in this phase generates real-time telemetry to demonstrate the platform's end-to-end detection capability in a controlled environment. The federated learning demonstration will collect model training metrics across 10 FL rounds from the simulated organizational clients, recording per-round accuracy, trust scores, and convergence data.

Evaluation (performance measurement): Data collection in this phase gathers both quantitative metrics and qualitative feedback to assess the artifact.

Quantitative data is collected automatically: scikit-learn's `classification_report` and `roc_auc_score` compute precision, recall, F1-score, and ROC-AUC on held-out test sets; Docker container statistics (``docker stats``) capture CPU and memory utilization; and FL convergence logs record global model vs. local model accuracy comparisons across rounds.

Qualitative data is collected through structured demonstration sessions where 3–5 participants investigate pre-loaded simulated APT alerts using the Wazuh Dashboard and attack graph visualization, capturing impressions on alert actionability, SHAP explanation clarity, and visualization usefulness.

3.6.2 Data Collection Tools

Table 1: Data Collection Tools

DSR Phase	Collection Method	Tools
Problem Identification	Literature review	IEEE Xplore, ACM Digital Library, Google Scholar, arXiv
	Threat pattern analysis	MITRE ATT&CK Navigator, ATT&CK Framework website
	Existing product analysis	Wazuh documentation, Darktrace/CrowdStrike/Vectra product literature
Design	APT attack dataset acquisition	Mordor Datasets (github.com/OTRF/mordor)
	Dataset preprocessing and partitioning	Python 3.11, pandas, NumPy, scikit-learn (<code>train_test_split</code>)
	Threat intelligence data caching	MITRE ATT&CK STIX data (JSON), PyMISP
	Real-World Data Collection	Python Scripts installed on Volunteered computers
Demonstration	Attack technique simulation	Atomic Red Team (<code>Invoke-AtomicTest</code> for T1003.001, T1021.002, T1059.001, T1550.002)
	Endpoint telemetry capture	Sysmon v15 (<code>sysmon_config.xml</code>), Windows Audit Policies
	Log collection and shipping	Wazuh Agent 4.7 (encrypted AES-256 transport to Wazuh Manager)
	Central log aggregation	Wazuh Manager 4.7 + Wazuh Indexer (OpenSearch)
	FL round metric recording	Flower (flwr) server logs, platform structured JSON logging
	Isolated test environment	Docker + Docker Compose (containerized lab network)
Evaluation	Model performance metrics	scikit-learn (<code>classification_report</code> , <code>roc_auc_score</code> , <code>confusion_matrix</code>), SHAP (<code>TreeExplainer</code>)
	Detection latency measurement	Platform structured JSON logs with correlation IDs (timestamp differencing)
	System resource profiling	<code>docker stats</code> (per-container CPU/memory), <code>GET /health</code> endpoint
	FL convergence analysis	matplotlib, seaborn (accuracy-per-round graphs)
	Usability scoring	SUS questionnaire (10-item standardized Likert scale)
	Qualitative feedback	Open-ended feedback forms, task completion sheets

3.7 System Requirements

3.7.1 Hardware Requirements

Table 2: Central Server Hardware Requirements

Component	Requirement
Processor	Intel Core i5 / AMD Ryzen 5, 4 cores
Memory (RAM)	16 GB (for FL simulation with 3 clients)
Storage (SSD)	100 GB (for extended log retention)
Network	>100 Mbps Ethernet

Table 3: Endpoints Hardware Requirements

Component	Requirement
OS	Windows 10/11 (64-bit)
RAM	4 GB minimum (Sysmon + Wazuh Agent use ~60MB combined)
Network	Connectivity to central server on port 1514

3.7.2 Software Requirements

Table 4: Central Server Software Requirements

Software	Version	Purpose
Ubuntu Server	22.04 LTS	Host operating system
Docker Engine	24.0+	Container runtime
Docker Compose	v2.20+	Multi-container orchestration
Python	3.11+	Platform runtime (inside containers)
Git	2.30+	Version control
14 Python packages	Latest	fastapi, xgboost, flwr, scikit-learn, shap, pydantic, httpx, networkx, pyvis, pyyaml, PyJWT, uvicorn, numpy, pandas

Table 5: Endpoints Software Requirements

Software	Version	Purpose
Windows	10/11 (64-bit)	Enterprise Operating System of choice
Sysmon	v15+	Deep endpoint monitoring
Wazuh Agent	4.7	Telemetry collection and shipping

CHAPTER FOUR

4 PROJECT TIMELINE AND BUDGET

4.1 Project Timeline

This section presents the detailed project schedule organized into sprints following Agile methodology. The project spans approximately 26 weeks from December 11, 2024 to June 10, 2025.

4.1.1 Gantt Chart - Project Timeline

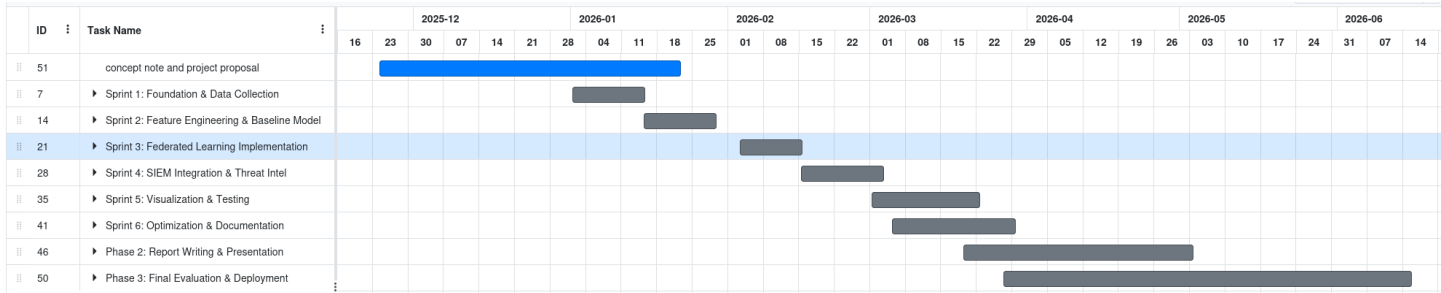


Figure 1: Project timeline

Critical Path: The critical path (longest sequence of dependent tasks) is:

Data Collection → Feature Engineering → ML Model Training → Federated Learning → SIEM Integration → Visualization → Testing → Documentation → Report Writing

Any delays in these activities will impact final delivery date.

4.2 Project Budget

The project budget accounts for all expenditures required to complete the system development, testing, and documentation.

4.2.1 Detailed Budget Breakdown

Table 6: Project Budget Breakdown

#	Item	Description	Quantity	Unit(TSH)	Total(TSH)
1	Compute Resources				200,000
1.1	Cloud Services	DigitalOcean/AWS for federated nodes simulation (6 months)	6 months	30,000	180,000
1.2	Domain & SSL	Optional external access domain registration	1	20,000	20,000
2	Data & Datasets				0
2.1	Public Datasets	Mordor (free downloads)	-	0	0
2.2	Synthetic Data Tools	Atomic Red Team (open-source)	-	0	0
3	Software & Tools				340,000
3.1	AI Tools Subscription	Claude AI for development assistance (optional)	6 months	55,000	330,000
3.2	MISP Hosting	Community instance (free) or minimal hosting	-	10,000	10,000
3.3	Development Tools	All open-source (Python, React, XGBoost, etc.)	-	0	0
4	Communication & Internet				60,000
4.1	Internet Data	Mobile data for off-campus work	6 months	10,000	60,000
5	Stationery & Prints				120,000
5.1	Printing Materials	Proposal, progress reports, questionnaires	Bulk	30,000	30,000
5.2	Final Report Printing	Final report printing (5 copies)	5 copies	15,000	75,000
5.3	Binding	Report binding and covers	5 copies	3,000	15,000
6	Data Collection				60,000
6.1	Questionnaire Print	Printing survey instruments	20 copies	500	10,000
6.2	Interview Incentives	Tokens for interview participants	5 people	10,000	50,000
7	Miscellaneous				50,000
7.1	Transportation	Travel to/from campus for meetings	6 months	5,000	30,000
7.2	Backup Storage	External HDD for data backup	1	20,000	20,000
	TOTAL				730,000

CHAPTER FIVE

5 PROJECT DESIGN

5.1 Introduction

The design phase of the threat hunting platform translates conceptual requirements into a structured, implementable architecture. This section outlines how the system is organized to achieve detection of CBLM APTs through the integration of Machine Learning technology (specifically XGBoost ML Classifier).

Additionally, this section details the structural and behavioral aspects of the system through diagrams such as achitecture overview, usecase diagrams, flowcharts, data flow diagrams (DFDs), sequence diagrams, contextual diagrams, state diagrams and deployment diagram. These representations provide a comprehensive understanding of how users such as platform Admin, Security operations center analyst interact with the system, and how data is securely processed and stored.

5.2 Functional and Non-Functional Requirements

5.2.1 Functional Requirements

Table 7: Threat Hunting Platform Functional Requirements

ID	Requirement
FR-01	The system shall collect security telemetry (Windows Security Events, Sysmon logs, PowerShell ScriptBlock logs) from corporate Windows endpoints via Wazuh Agent, with PII anonymization at the collection point, deployable through a single automated script.
FR-02	The system shall ingest normalized events from Wazuh SIEM via its REST API at configurable intervals, validating and sanitizing all input data before processing.
FR-03	The system shall extract behavioral features from security events through a composable pipeline of pluggable extractors covering authentication, process execution, temporal, behavioral, and network patterns.
FR-04	The system shall detect credential-based lateral movement (Pass-the-Hash, Kerberos abuse, SMB lateral movement) using a trained XGBoost classifier, producing detections with confidence scores, severity levels, and SHAP-based feature explanations.
FR-05	The system shall support a plugin-based detection engine where new detection models can be added by creating a single Python file implementing a standard interface plus a YAML configuration entry, with zero changes to core platform code.
FR-06	The system shall enrich detections with MITRE ATT&CK technique and tactic mappings (T1003, T1021, T1550, T1078) and generate context-specific recommendations for analysts.
FR-07	The system shall manage alert lifecycle (scoring, deduplication, persistence, status tracking from open through acknowledged to resolved/closed) and publish enriched alerts back to Wazuh Dashboard for unified SOC visibility.
FR-08	The system shall construct and render interactive attack path graphs from correlated

ID	Requirement
	detections, visualizing lateral movement chains as directed graphs with severity-coded nodes and MITRE technique-labeled edges.
FR-09	The system shall implement privacy-preserving federated learning using Flower framework with XGBoost federated bagging, differential privacy ($\epsilon=1.0$), and trust-based contribution validation, enabling cross-organization model improvement without sharing raw security data.
FR-10	The system shall provide a REST API with endpoints for alert querying, model status, health monitoring, and model retraining, with auto-generated interactive documentation.

5.2.2 Non-functional Requirements

Table 8: Threat Hunting Platform Non-functional Requirements

ID	Requirement
NFR-01	Performance: The system shall detect threats within 2 minutes (95th percentile) from event occurrence to alert publication, processing up to 10,000 events per polling cycle.
NFR-02	Accuracy: The lateral movement detection model shall achieve precision $\geq 85\%$, recall $\geq 90\%$, ROC-AUC ≥ 0.93 , and false positive rate $< 5\%$ on held-out test data.
NFR-03	Reliability: The system shall maintain $> 99\%$ uptime, gracefully handle Wazuh SIEM downtime via circuit breaker with automatic recovery, persist alert state across restarts, and isolate individual module failures from crashing the platform.
NFR-04	Scalability: The system shall support 10–100+ monitored endpoints, 2–5 federated learning organizations, with per-container resource limits preventing host exhaustion.
NFR-05	Usability: The system shall achieve SUS score ≥ 70 , with $\geq 80\%$ of evaluation participants able to complete an alert investigation task, and alerts rated actionable by $\geq 75\%$ of participants. AI-enriched alerts shall appear in the existing Wazuh Dashboard without requiring a separate tool.
NFR-06	Maintainability: The system shall be deployable via a single <code>docker compose up -d</code> command, with all parameters configurable via YAML, all modules independently testable with no circular dependencies, and structured JSON logging with correlation IDs for end-to-end tracing.
NFR-07	Portability: The platform shall run on any Docker-capable Linux server with no host-level dependencies beyond Docker and Docker Compose, using only open-source software with zero licensing cost. Endpoint collection shall support Windows 10/11.

5.2.3 Security Requirements

Table 9: Threat Hunting Platform Security Requirements

ID	Requirement
SR-01	Authentication & Authorization: All API requests shall be authenticated via JWT tokens or API keys, with role-based access control enforcing four roles (viewer, analyst, operator, admin) across 11 permissions. Unauthenticated requests receive HTTP 401; unauthorized requests receive HTTP 403.

ID	Requirement
SR-02	Data Protection: All endpoint-to-server communication shall be encrypted (AES-256 via Wazuh protocol), all API communication shall use HTTPS, and all credentials shall be managed via Docker Secrets — never stored in environment variables, code, or container images.
SR-03	Model Integrity: Every XGBoost model file shall be HMAC-SHA256 signed at training time and verified at load time. Tampered models shall be refused with an immediate administrator alert. Model versioning shall support rollback to previous verified versions.
SR-04	Audit Trail: All security-relevant actions (model retraining, threshold changes, alert acknowledgments, FL rounds, authentication failures) shall be logged to a hash-chained, append-only audit trail where each entry's hash includes the previous entry's hash, enabling tamper detection through chain integrity verification.
SR-05	Federated Learning Privacy: No raw security telemetry shall cross organizational boundaries during federated learning. Differential privacy (Laplace noise, $\epsilon=1.0$) shall be applied to all model updates before transmission. FL clients with trust scores below 0.3 (from repeated validation failures) shall be blocked from participation.
SR-06	Container Hardening & Input Sanitization: All Docker containers shall run with read-only filesystems, non-root users, all Linux capabilities dropped, and no-new-privileges enforced. The FL network shall be isolated from external access. All ingested data shall pass 6-layer input validation rejecting injection patterns, malformed IPs, out-of-bounds timestamps, and oversized payloads.

5.3 System Design

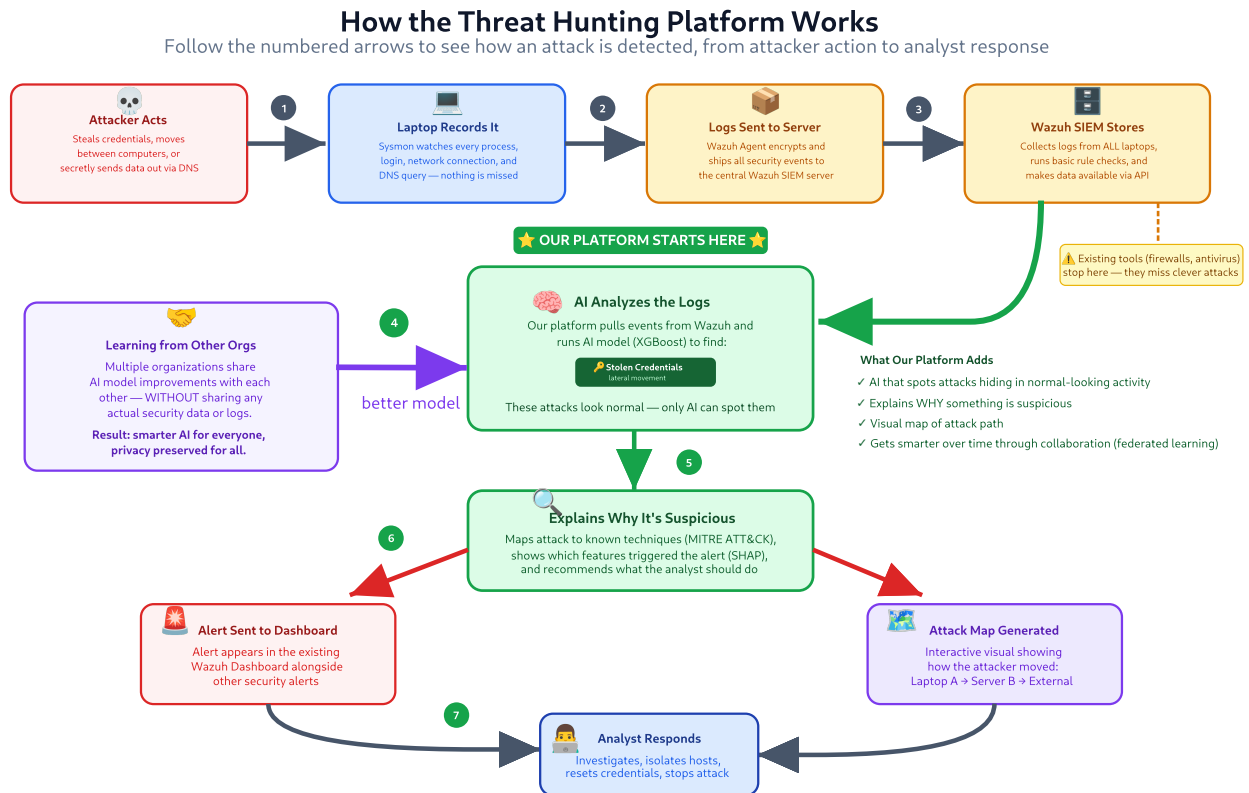


Figure 2: Threat Hunting Platform Architecture Design

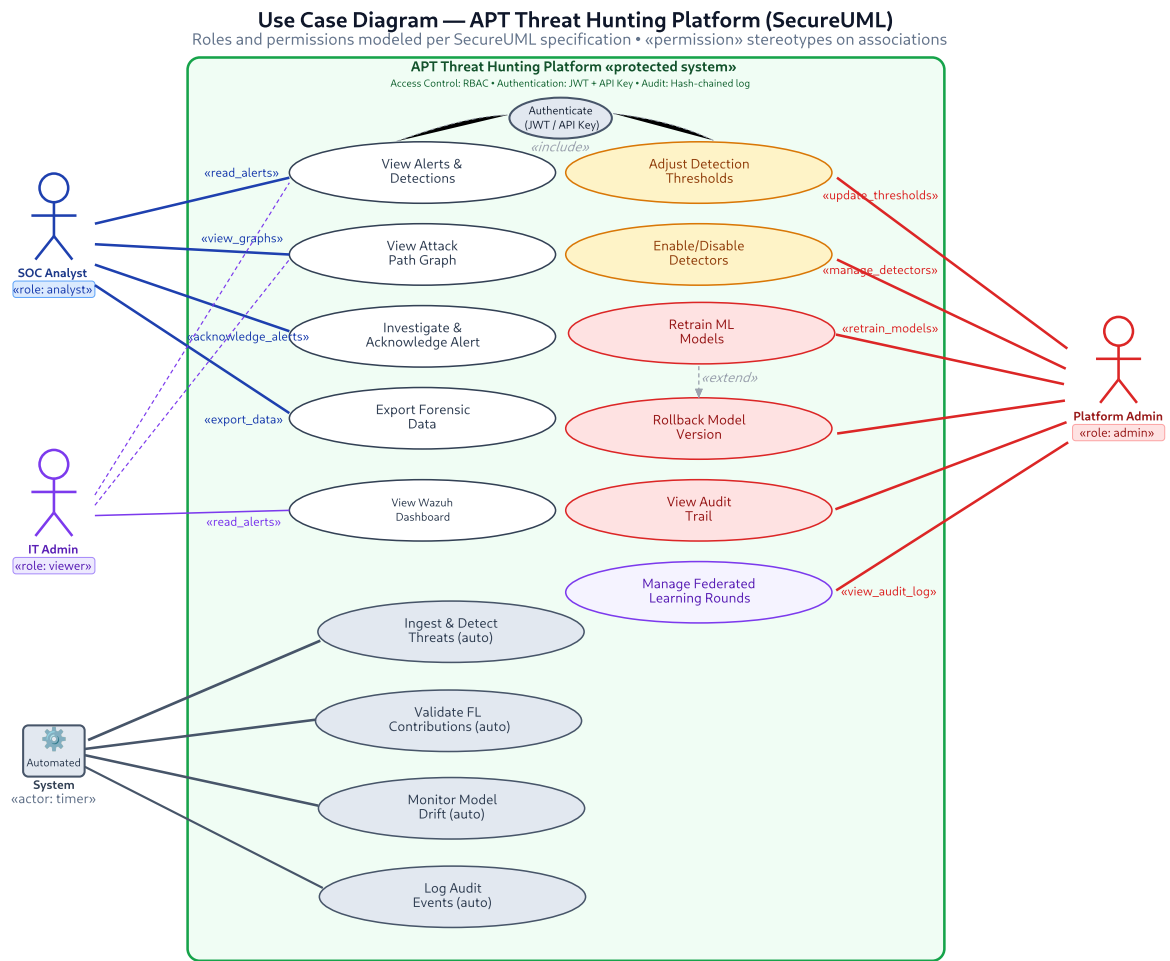


Figure 3: Threat Hunting Platform UseCase Diagram

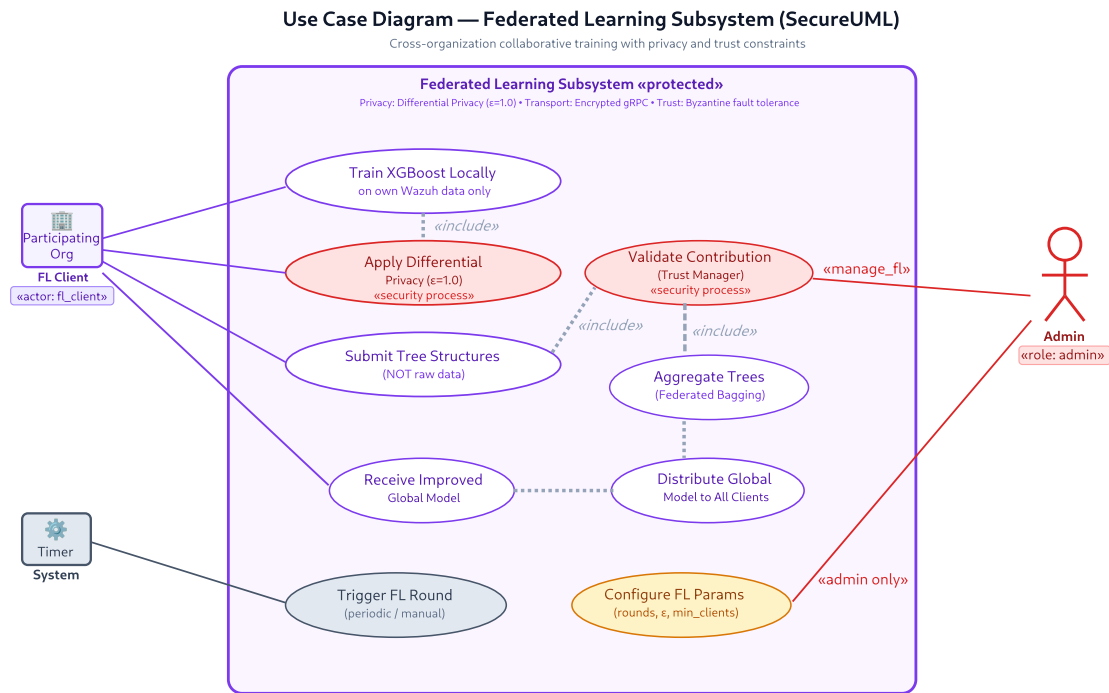


Figure 4: Threat Hunting Platform Usecase Diagram for Federated Learning Subsystem

Sequence Diagram — API Authentication & RBAC (SecureUML)

JWT + API Key authentication with role-based access control and audit logging

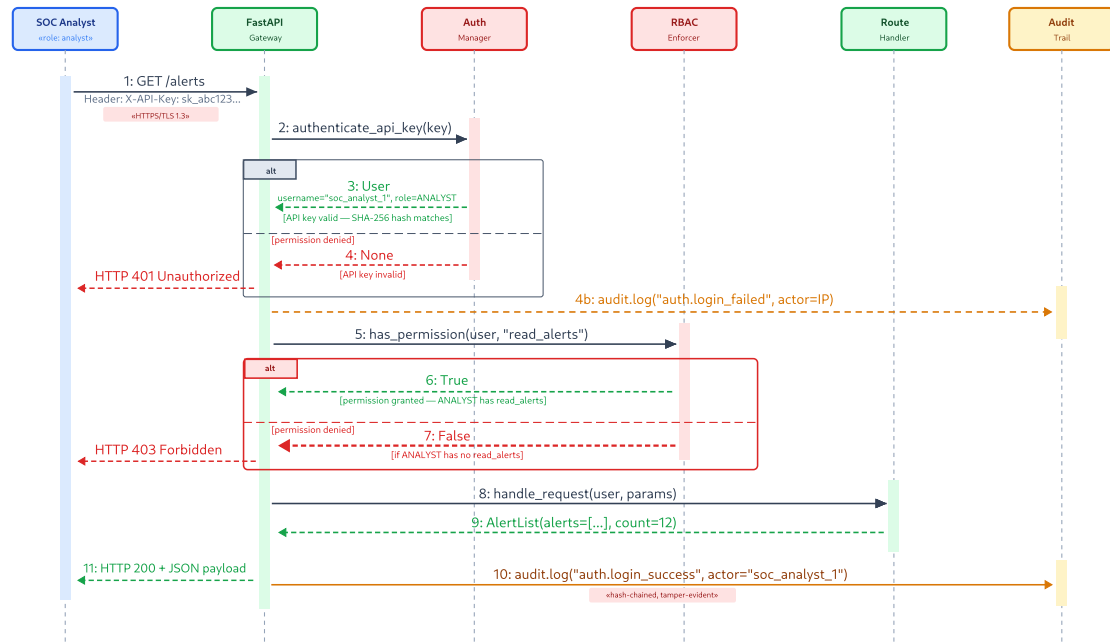


Figure 5: Threat Hunting Platform Sequence Diagram (API auth & RBAC)

Sequence Diagram — Threat Detection Pipeline

From endpoint telemetry to SOC analyst alert • Security checkpoints annotated

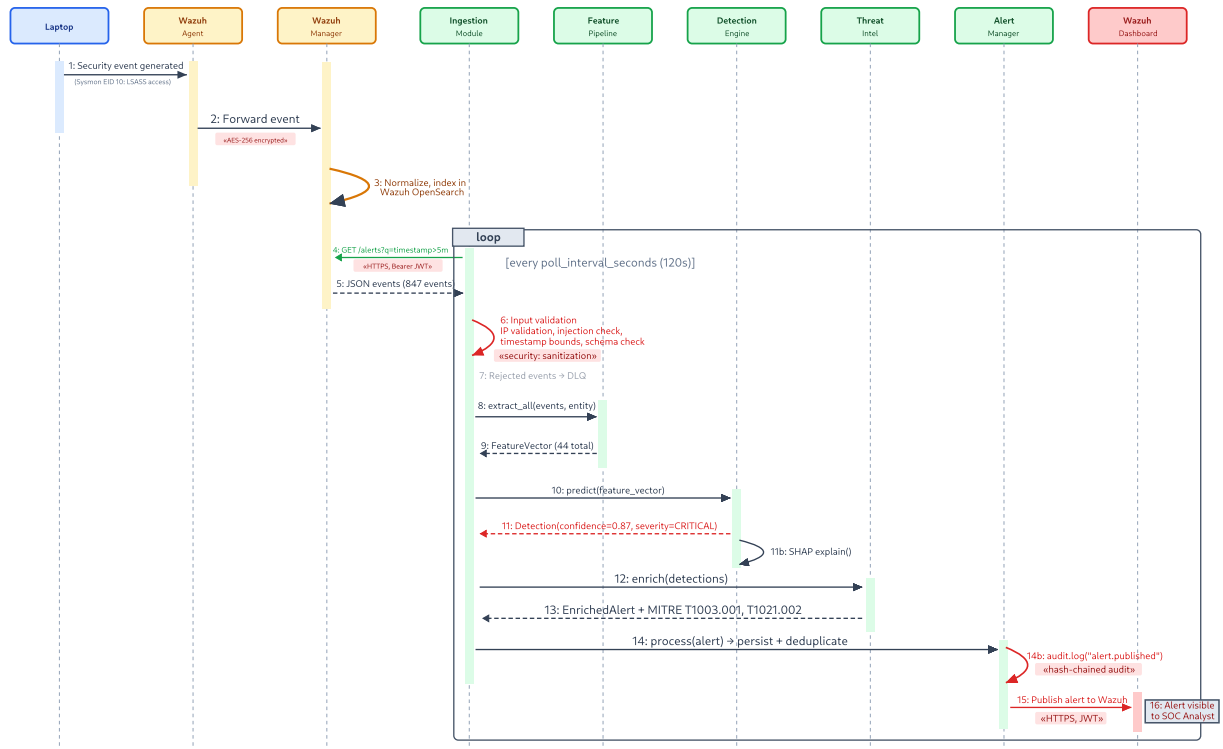


Figure 6: Threat Hunting Platform Sequence Diagram (threat detection pipeline)

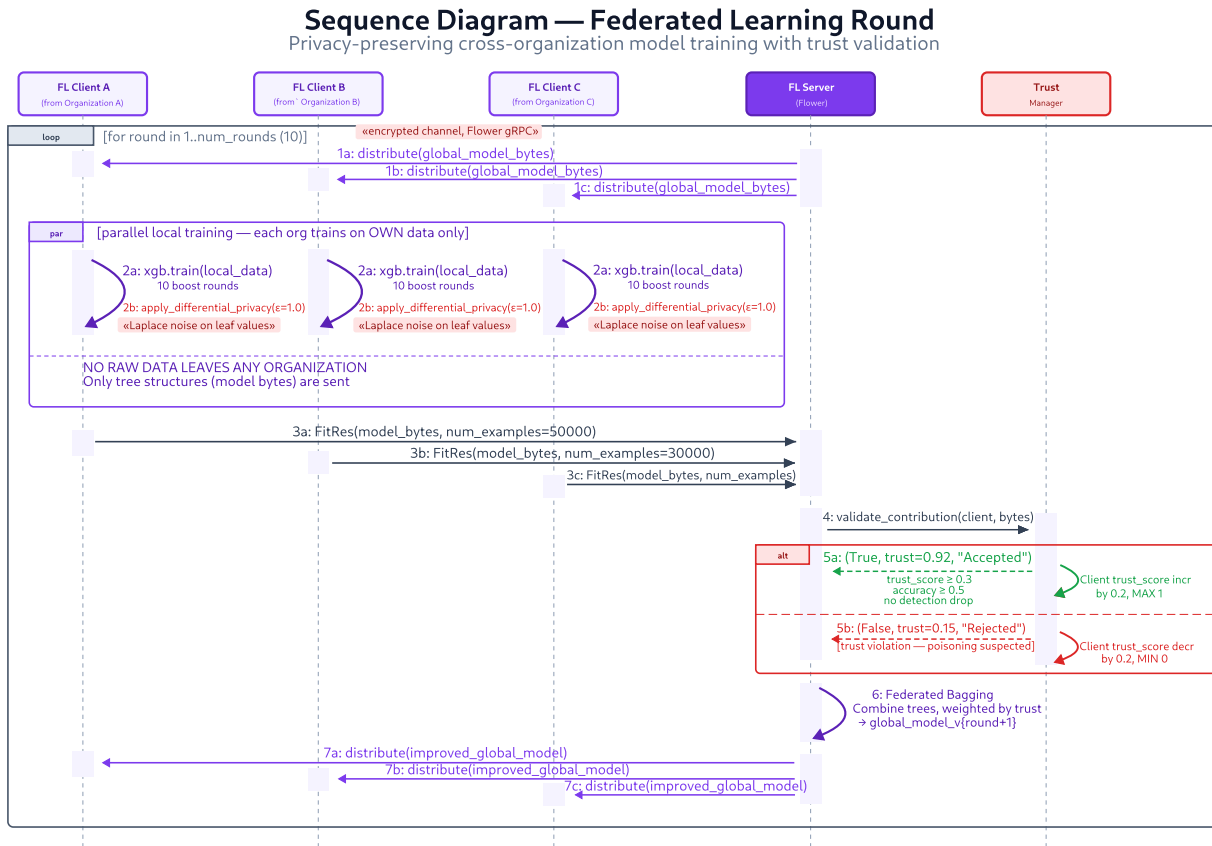


Figure 7: Threat Hunting Platform Sequence Diagram (federated learning)

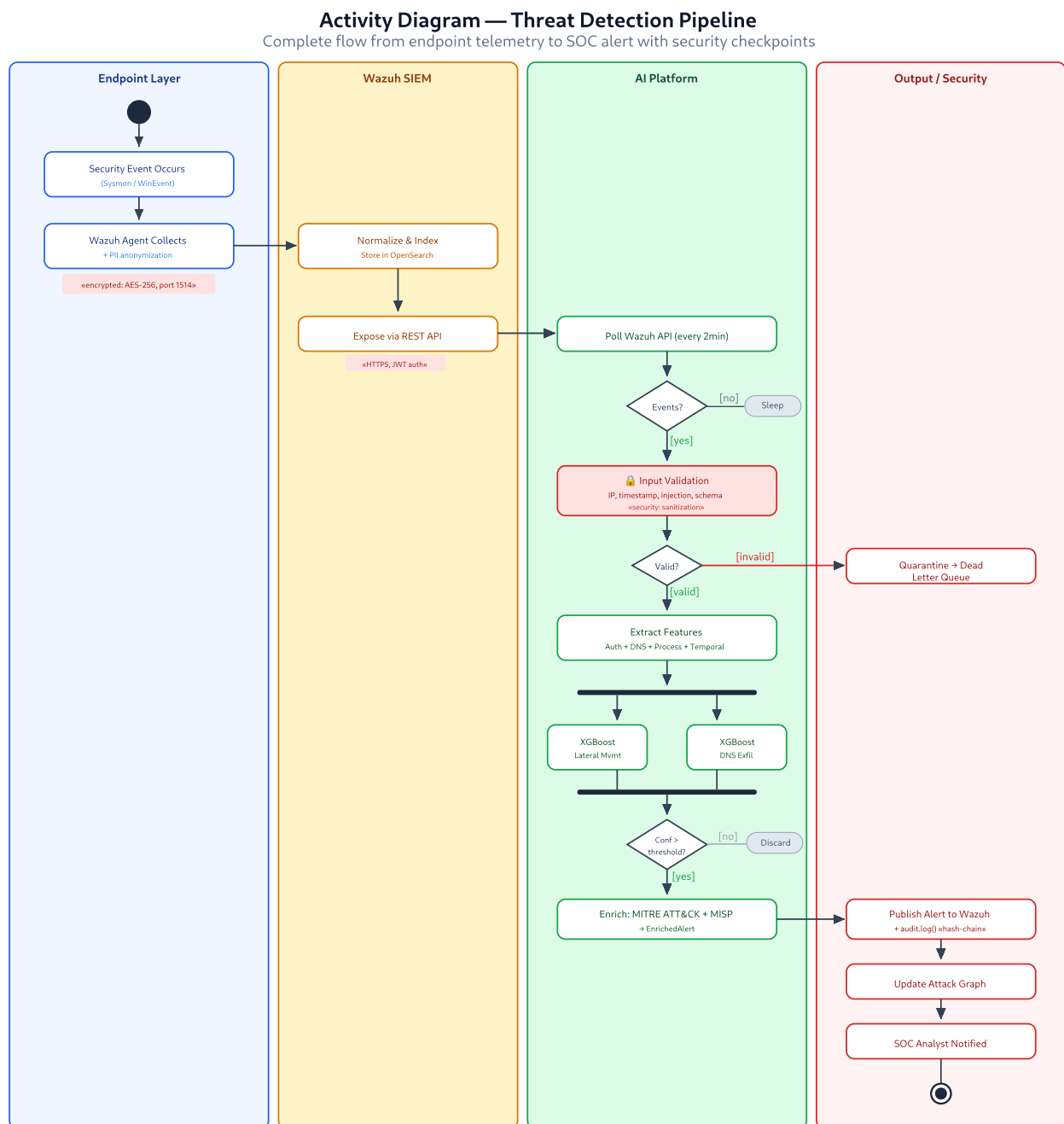


Figure 8: Threat Hunting Platform Activity Diagram (detection engine)

Activity Diagram — Alert Investigation Workflow

SOC Analyst and IT Admin investigation process with RBAC enforcement

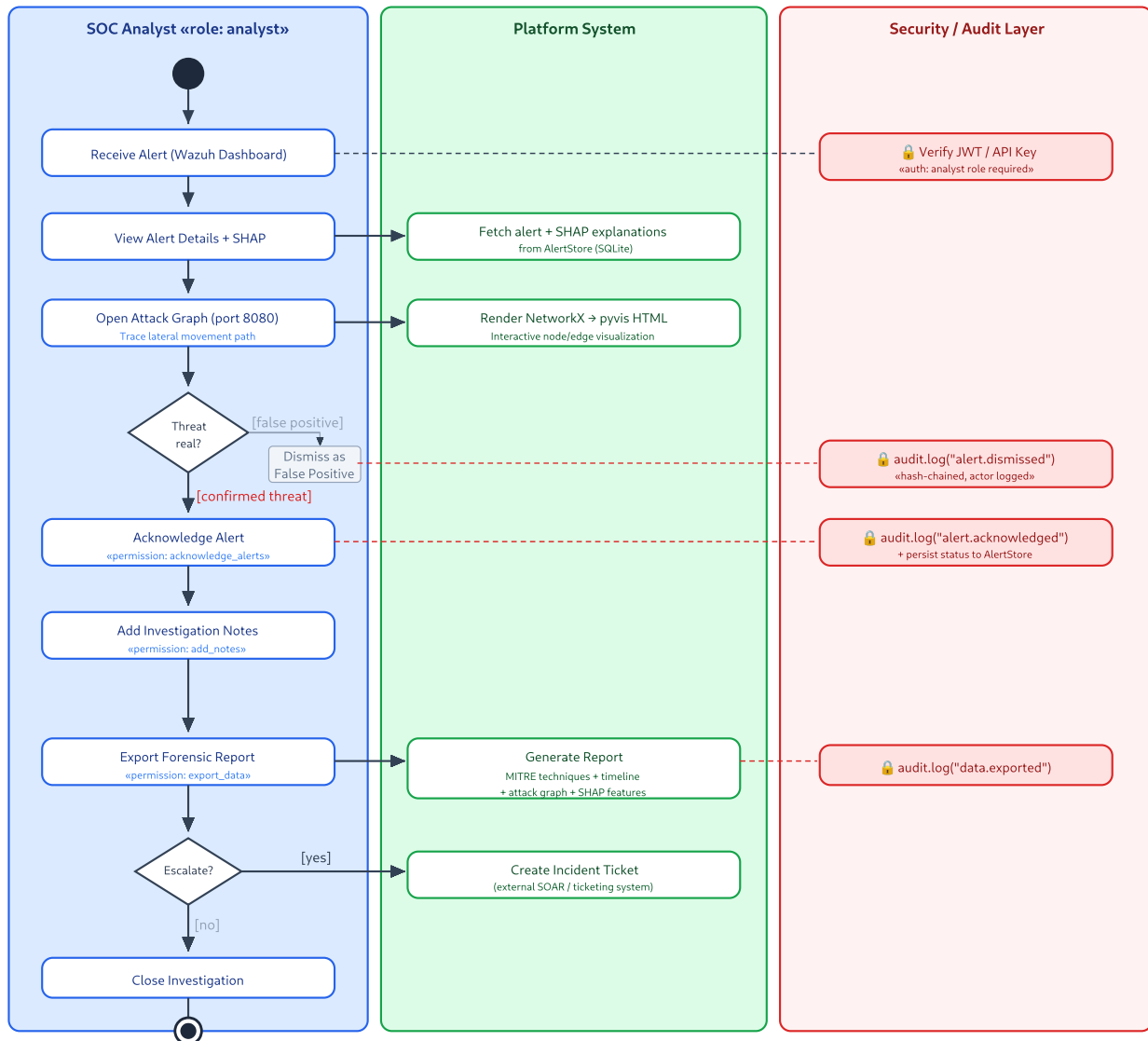


Figure 9: Threat Hunting Platform Activity Diagram (alert propagation)

Context Diagram (DFD Level 0) — APT Threat Hunting Platform
SecureUML: All external data flows annotated with security constraints

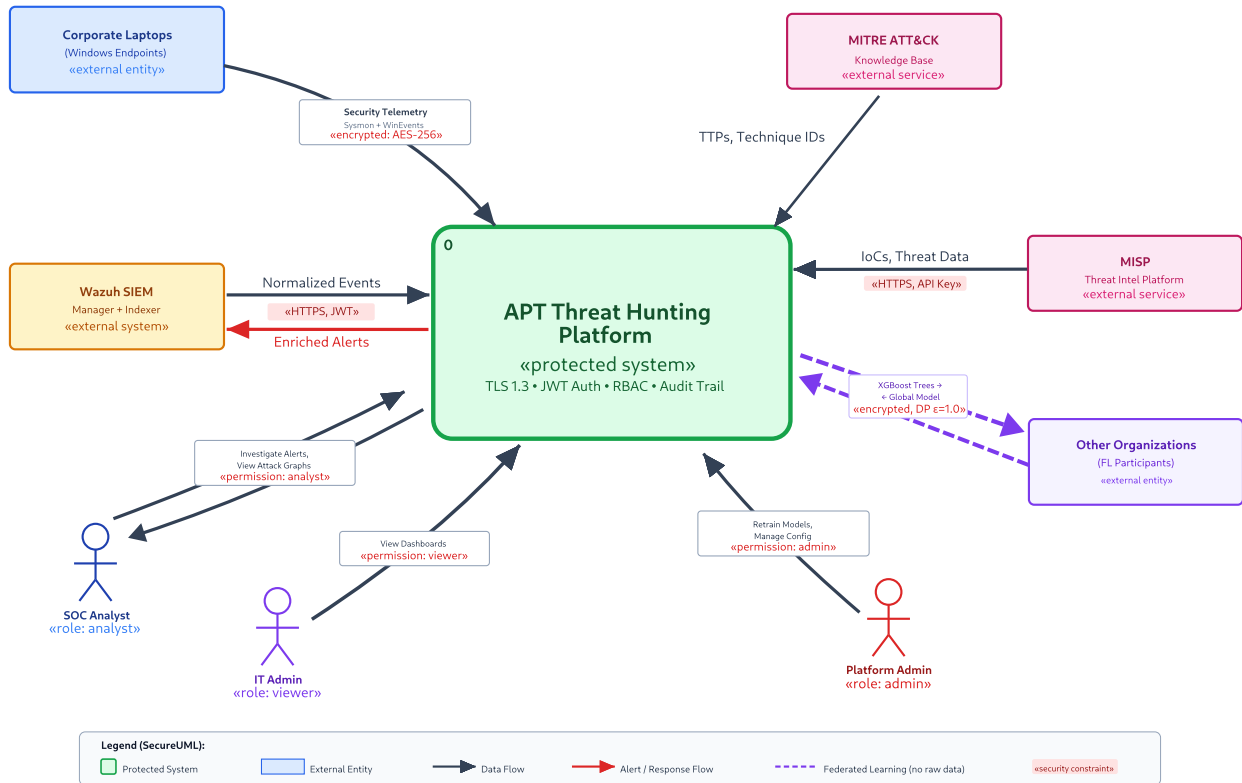


Figure 10: Threat Hunting Platform Context Diagram

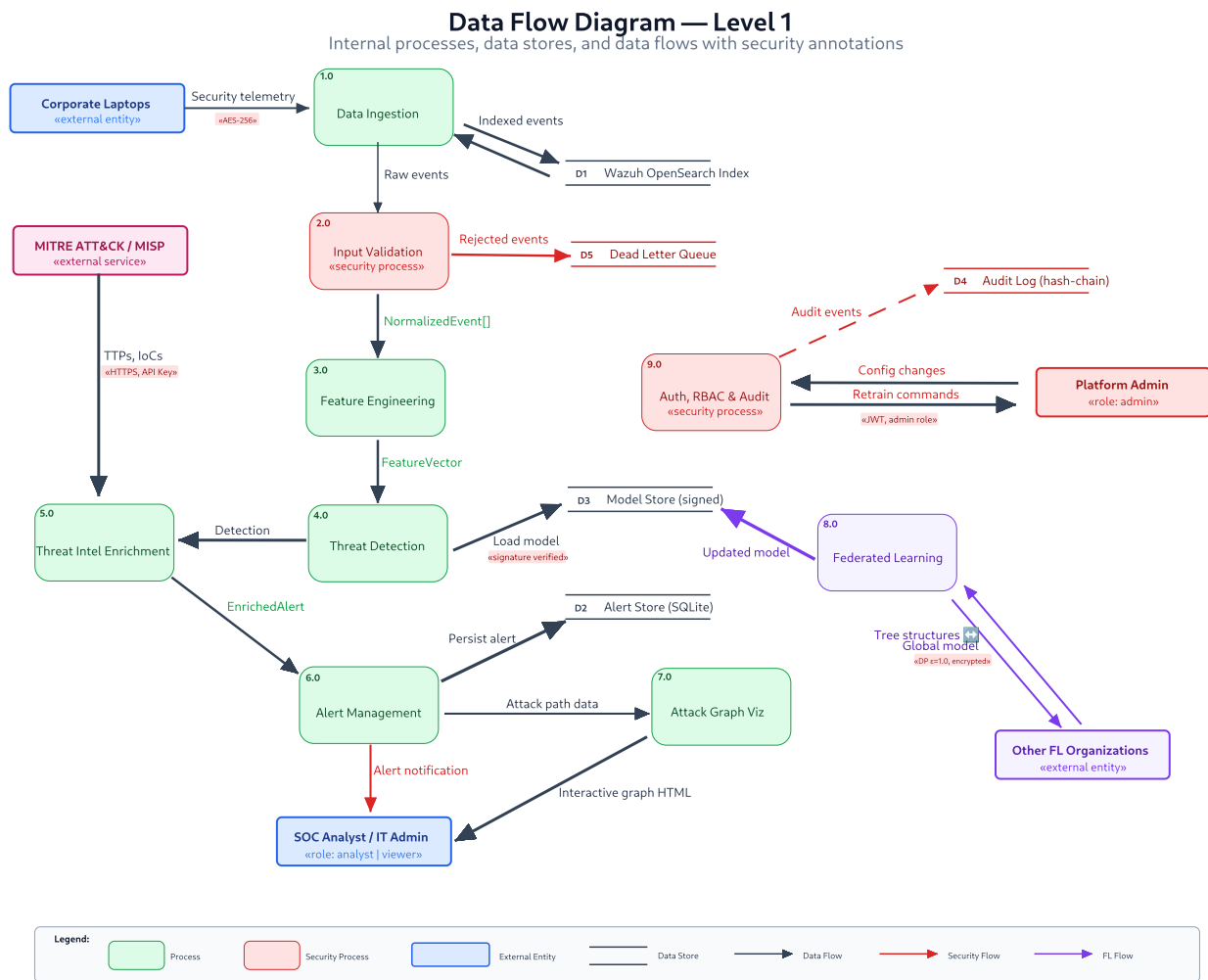


Figure 11: Threat Hunting Platform DFD level 1

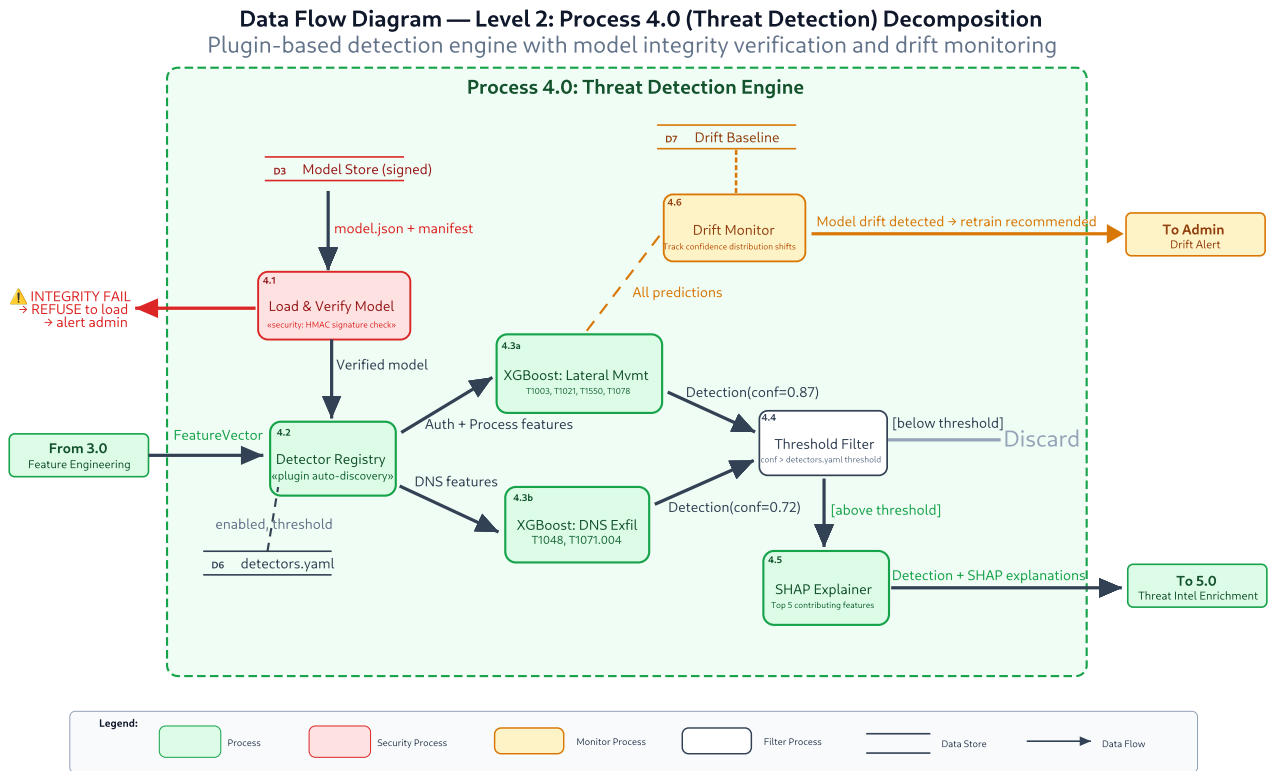


Figure 12: Threat Hunting Platform DFD level 2 (threat detection)

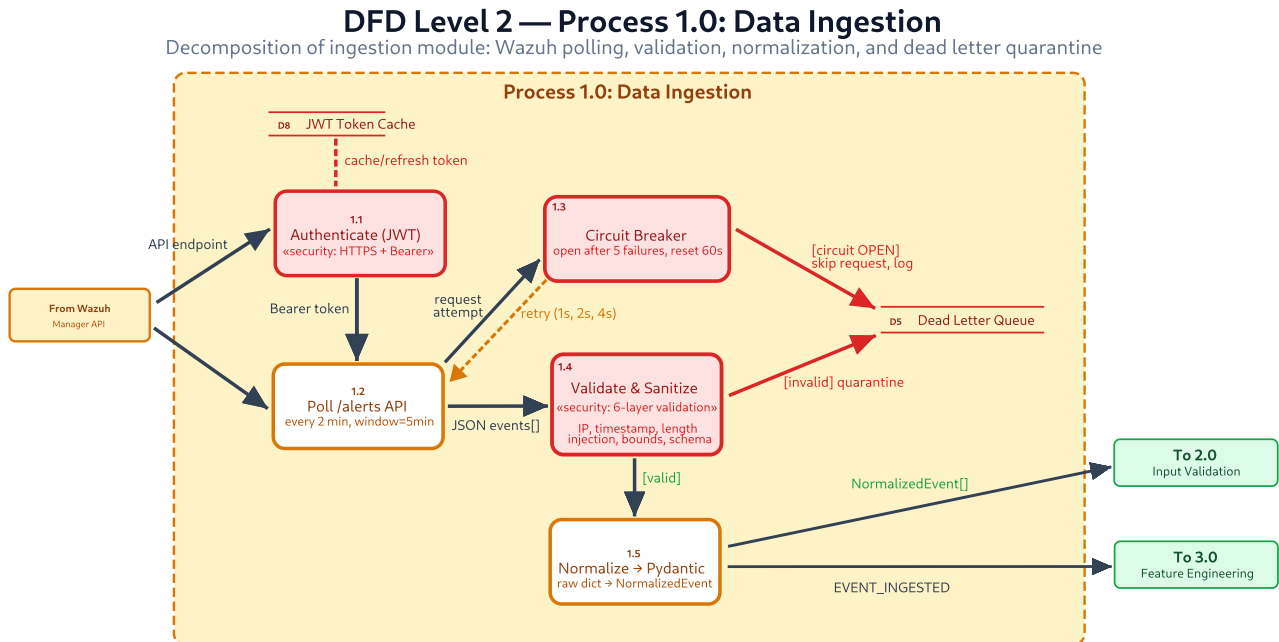


Figure 13: Threat Hunting Platform DFD level 2 (data ingestion)

DFD Level 2 — Process 3.0: Feature Engineering

Composable extractor pipeline with 6 pluggable extractors, prefix namespacing, and LRU caching

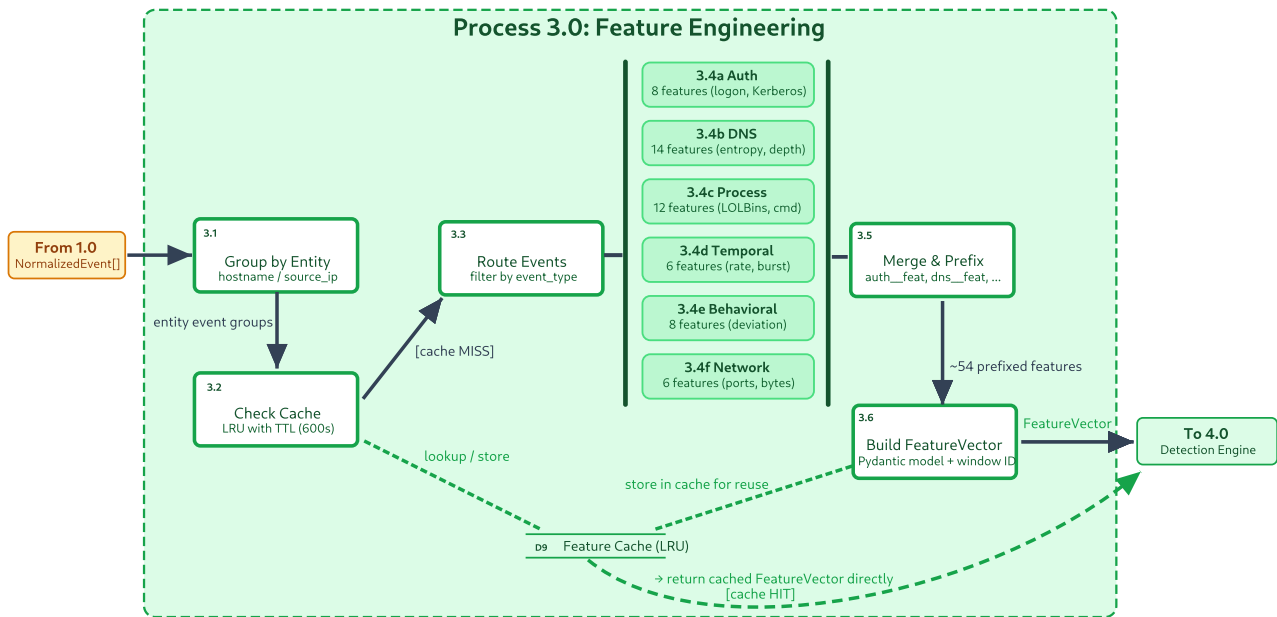


Figure 14: Threat Hunting Platform DFD level 2 (feature engineering)

DFD Level 2 — Process 5.0: Threat Intelligence Enrichment

MITRE ATT&CK mapping, MISP IoC correlation, severity escalation, and response action generation

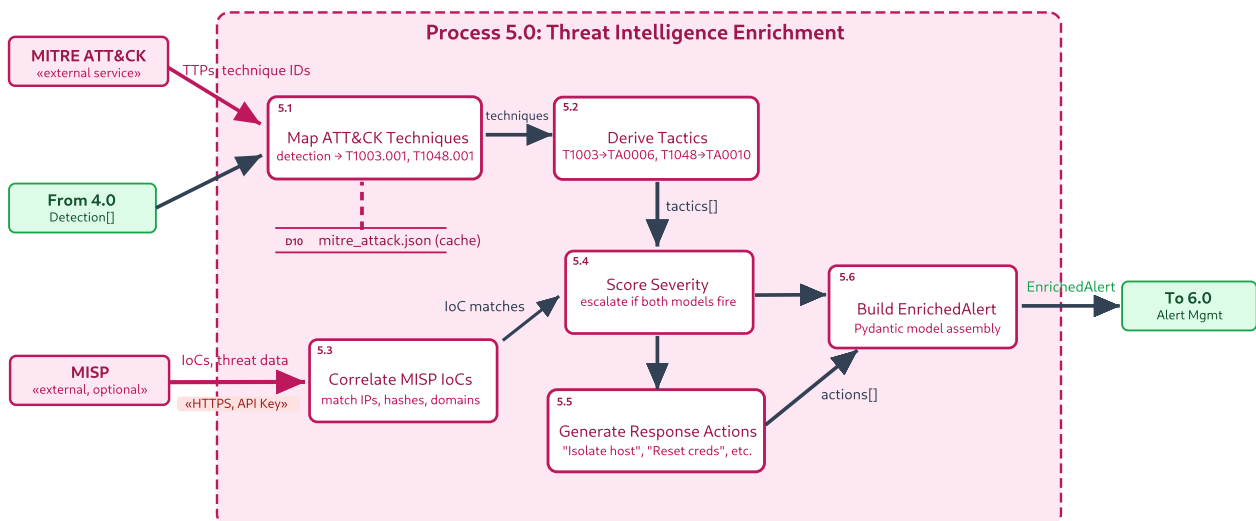


Figure 15: Threat Hunting Platform DFD level 2 (intelligence enrichment)

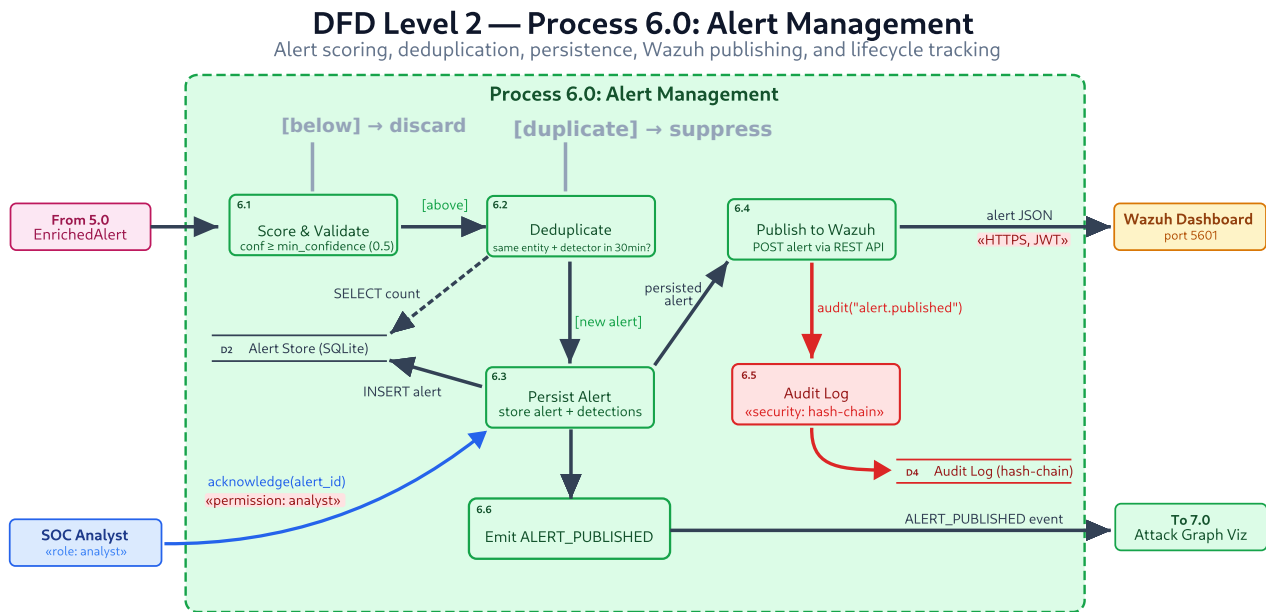


Figure 16: Threat Hunting Platform DFD level 2 (alert management)

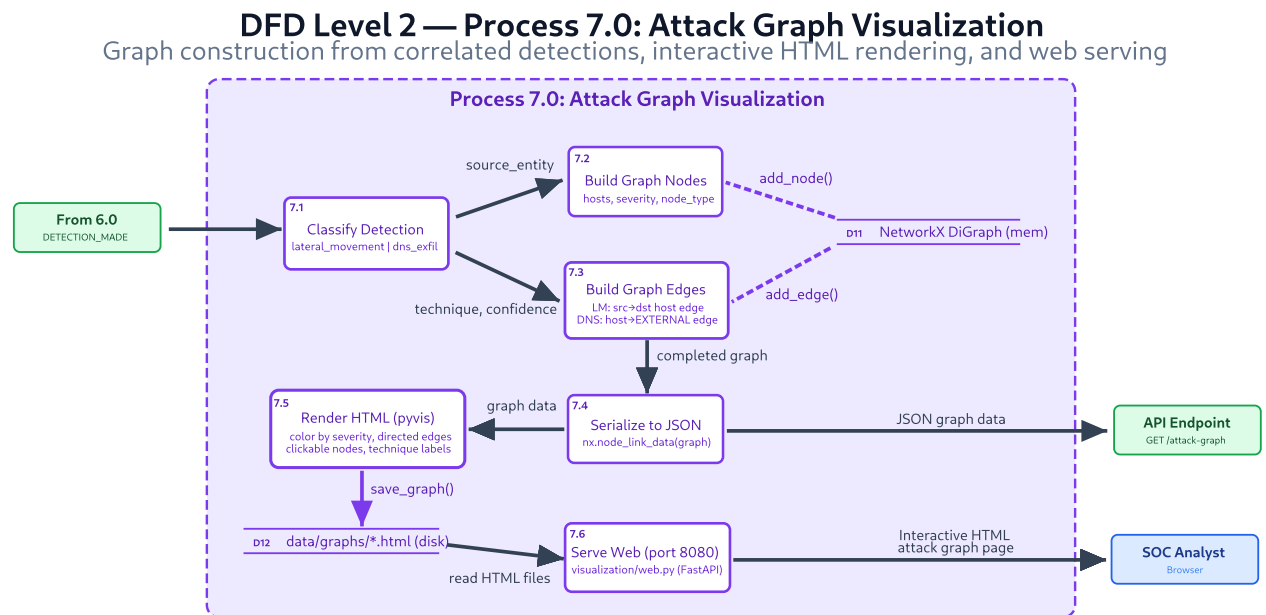


Figure 17: Threat Hunting Platform DFD level 2 (attack graph visualization)

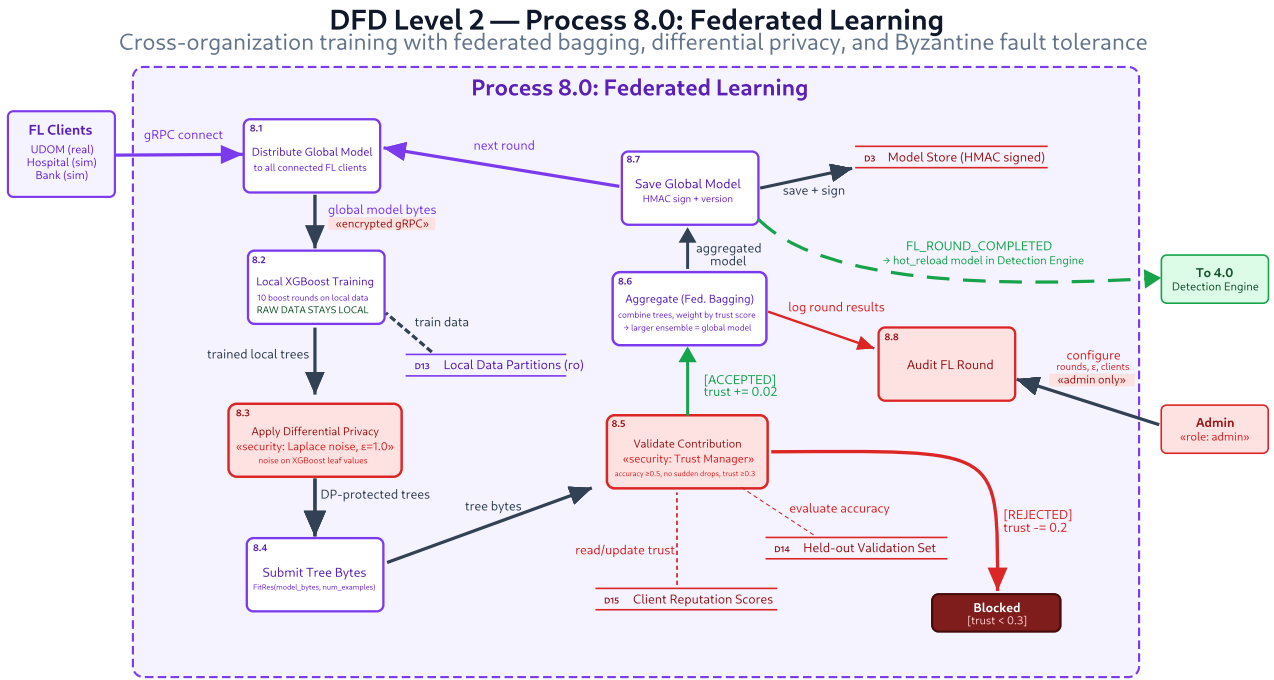


Figure 18: Threat Hunting Platform DFD level 2 (federated learning)

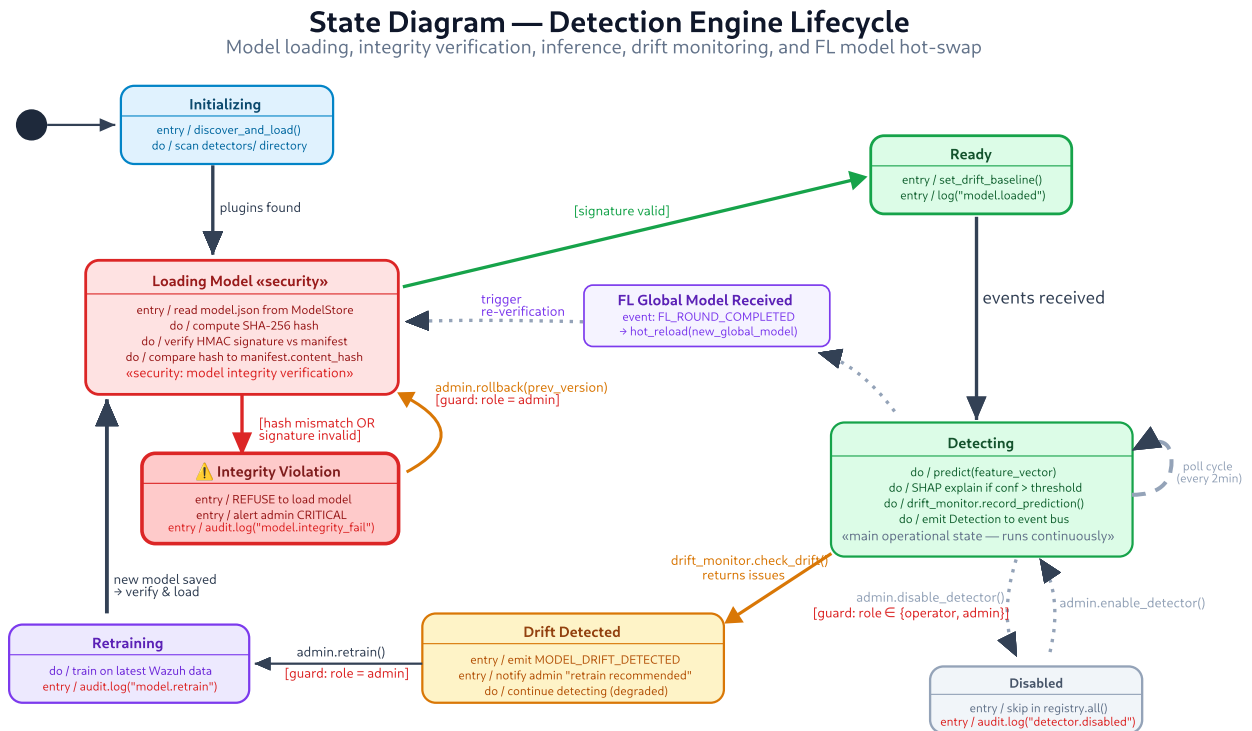


Figure 19: Threat Hunting Platform state diagram (detection engine)

State Diagram — Federated Learning Client
Per-organization FL client lifecycle with privacy and trust constraints

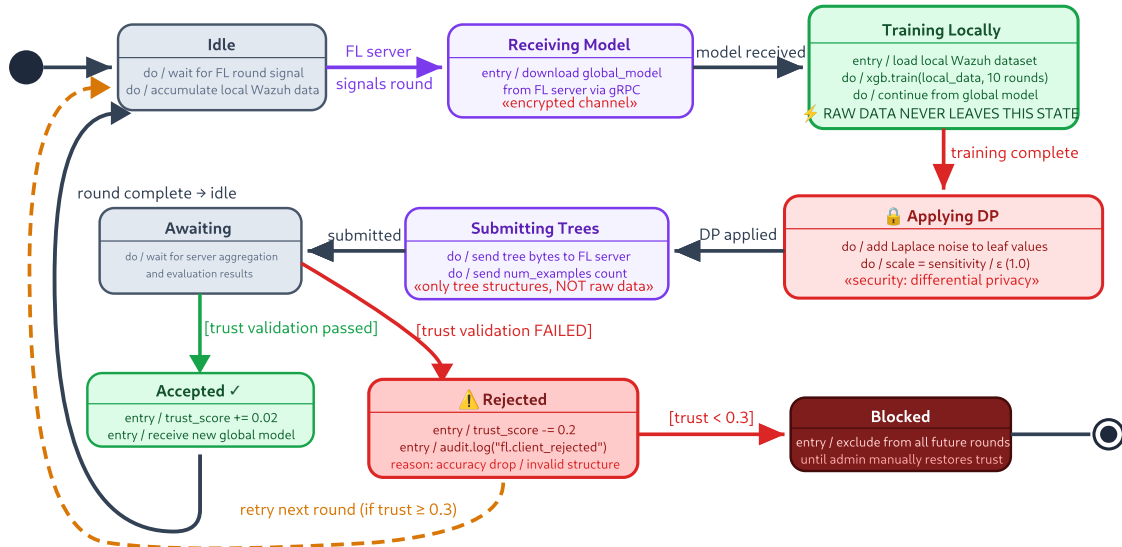


Figure 20: Threat Hunting Platform state diagram (federated learning client)

State Diagram — Alert Lifecycle
States, transitions, guards, and SecureUML permission constraints on state changes

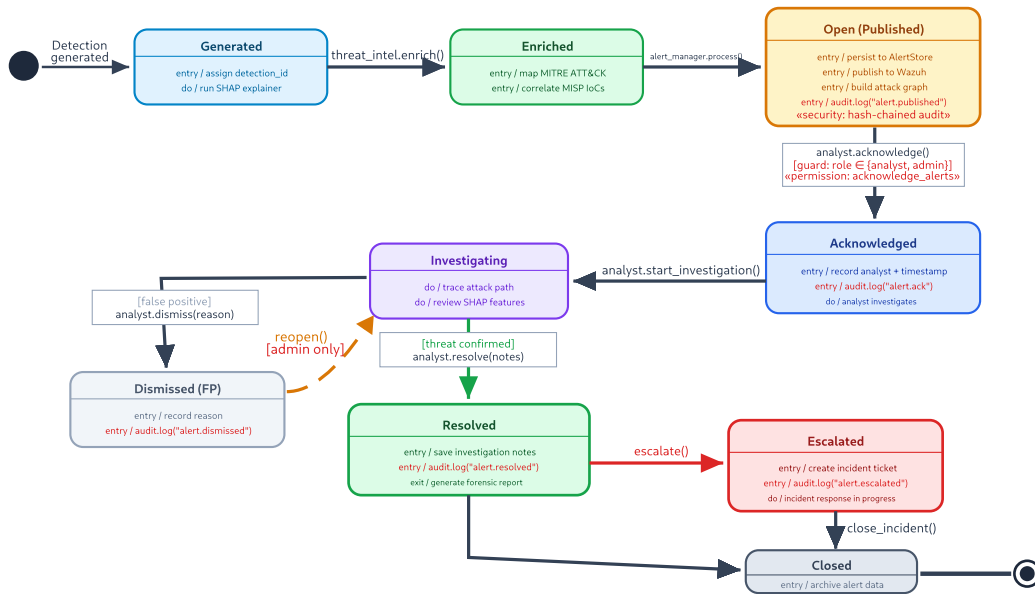


Figure 21: Threat Hunting Platform state diagram (alert lifecycle)

5.4 Achievements so far

5.4.1 Objective 1: Design

Table 10: Threat Hunting Platform objective 1 progress

S/N	Activity	Expected Output	Progress (%)
1.1	Define shared Pydantic data contracts	shared/schemas.py with NormalizedEvent, FeatureVector, Detection, EnrichedAlert	100%
1.2	Define abstract interfaces for detectors and extractors	shared/interfaces.py (BaseDetector, BaseFeatureExtractor, HealthCheckable)	100%
1.3	Implement plugin-based detector registry with auto-discovery	detection/registry.py scanning detectors/directory	100%
1.4	Design RBAC permission model (4 roles, 11 permissions)	shared/security.py (AuthManager, Role, PERMISSIONS matrix)	100%
1.5	Implement HMAC model integrity store design	detection/model_store.py (signing, verification, versioning, rollback)	100%
1.6	Design async event bus architecture	shared/events.py (pub/sub with 7 event types, error isolation)	100%
1.7	Design centralized config system	shared/config.py (YAML loading, env-var + Docker secret resolution)	100%
1.8	Design Docker Compose deployment topology	docker-compose.yml (10 containers, 3 networks, secrets, security hardening)	100%
1.9	Create YAML configuration files	config/platform.yaml, wazuh.yaml, detectors.yaml, federated.yaml, security.yaml	100%
1.10	Produce UML/design diagrams with SecureUML annotations	15 SVG diagrams (use case, sequence, activity, state, DFD, deployment)	100%

5.4.2 Objective 2: Detection Engine Implementation

Table 11: Threat Hunting Platform objective 2 progress

S/N	Activity	Expected Output	Progress (%)
2.1	Acquire and preprocess Mordor dataset	Cleaned, labeled lateral movement events (~500K)	40%
2.2	Create Sysmon APT-optimized configuration	scripts/sysmon_config.xml (EIDs 1,3,7,8,10,13,22,25)	50%
2.3	Create endpoint deployment script	scripts/deploy_endpoint.ps1 (Sysmon + audit policies + Wazuh Agent)	50%
2.4	Implement Wazuh API connector with circuit breaker	ingestion/wazuh_connector.py (retry, backoff, circuit breaker)	50%
2.5	Implement 6-layer input validation	ingestion/preprocessor.py +	50%

S/N	Activity	Expected Output	Progress (%)
	preprocessor	ingestion/dead_letter.py	
2.6	Implement 6 feature extractors	features/auth, dns, process, temporal, behavioral, network	35%
2.7	Implement composable feature pipeline with caching	features/pipeline.py + features/cache.py	50%
2.8	Train XGBoost lateral movement classifier	detection/models/lateral_movement_v1.json	30%
2.9	Implement lateral movement detector plugin	detection/detectors/lateral_movement.py	50%
2.10	Implement SHAP explainability for detections	detection/explainer.py (top contributing features per detection)	50%
2.11	Implement drift monitor	detection/drift_monitor.py (confidence/rate shift tracking)	50%
2.12	Hyperparameter tuning (GridSearchCV)	Optimized model parameters for both classifiers	20%

5.4.3 Objective 3: Platform Subsystems Implementation

Table 12: Threat Hunting Platform objective 3 progress

S/N	Activity	Expected Output	Progress (%)
3.1	Implement Flower FL server with bagging strategy	federated/server.py + federated/aggregation.py	50%
3.2	Implement FL client (local XGBoost training)	federated/client.py (trains locally, sends tree bytes only)	50%
3.3	Implement differential privacy module	federated/privacy.py (Laplace noise on leaf values, $\epsilon=1.0$)	50%
3.4	Implement trust manager with reputation scoring	federated/trust.py (validation, accuracy checks, blocking)	50%
3.5	Partition datasets into 3 organizational subsets	training/data/partition_1,2,3 (UDOM, Hospital, Bank)	35%
3.6	Run federated training rounds and compare models	Convergence graphs + global vs. local accuracy comparison	20%
3.7	Implement MITRE ATT&CK enrichment	threat_intel/enricher.py (technique + tactic mapping, response actions)	50%
3.8	Implement alert engine with SQLite persistence	alert_manager/alert_engine.py + alert_manager/store.py	50%
3.9	Implement Wazuh alert publisher	alert_manager/wazuh_publisher.py (bidirectional SIEM integration)	45%

S/N	Activity	Expected Output	Progress (%)
3.10	Implement attack graph builder	visualization/graph_builder.py (NetworkX directed graph)	50%
3.11	Implement interactive graph renderer	visualization/renderer.py + visualization/web.py (pyvis HTML, port 8080)	50%
3.12	Implement API authentication middleware (JWT + RBAC)	api/middleware.py (4 roles, 11 permissions enforced per endpoint)	50%
3.13	Implement hash-chained audit trail	observability/audit.py (append-only SQLite, tamper-evident chain)	50%
3.14	Implement health check protocol	shared/health.py + api/routes/health.py (aggregated component status)	50%
3.15	Implement async event bus	shared/events.py (pub/sub, error isolation, event replay)	50%
3.16	Implement structured JSON logging with correlation IDs	shared/logging.py (machine-parseable, end-to-end tracing)	50%
3.17	Implement FastAPI server with detection loop	api/main.py (lifespan startup, background async polling loop)	50%
3.18	Finalize Docker Compose with security hardening	docker-compose.yml (read-only FS, non-root, cap_drop, resource limits, network segmentation)	50%

5.5 Current Progress Status

The project is now moving into the implementation stage.

Current efforts are focused on detection engine components; Create endpoint deployment script, Implement Wazuh API connector with circuit breaker, input validation preprocessor, feature extractors, composable feature pipeline with caching. Only then can we Train XGBoost lateral movement classifier and Implement lateral movement detector plugin.

REFERENCES

- Apache Software Foundation. (2020). Apache Metron. <https://metron.apache.org/>
- Axelsson, S. (2000). The base-rate fallacy and the difficulty of intrusion detection. *ACM Transactions on Information and System Security*, 3(3), 186-205.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Box, G. E., & Jenkins, G. M. (1976). *Time series analysis: forecasting and control*. Holden-Day.
- Caltagirone, S., Pendergast, A., & Betz, C. (2013). *The Diamond Model of Intrusion Analysis*. Center for Cyber Intelligence Analysis and Threat Research.
- Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 1-58.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
- Chuvakin, A., & Peterson, G. (2010). How to Do Application Logging Right. *IEEE Security & Privacy*, 8(4), 82-85.
- CrowdStrike. (2024). Falcon Platform Documentation. <https://www.crowdstrike.com/>
- Darktrace. (2023). Enterprise Immune System: Self-Learning AI for Cyber Defense. <https://darktrace.com/>
- Denning, D. E. (1987). An intrusion-detection model. *IEEE Transactions on Software Engineering*, 13(2), 222-232.
- Dhaliwal, S. S., Nahid, A. A., & Abbas, R. (2018). Effective intrusion detection system using XGBoost. *Information*, 9(7), 149.
- Domingos, P. (2012). A few useful things to know about machine learning. *Communications of the ACM*, 55(10), 78-87.
- Dwork, C. (2006). Differential privacy. *International Colloquium on Automata, Languages, and Programming*, 1-12. Springer.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189-1232.
- Gartner. (2023). *Market Guide for Security Information and Event Management*. Gartner Research.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.
- Hutchins, E. M., Cloppert, M. J., & Amin, R. M. (2011). Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. *Leading Issues in Information Warfare & Security Research*, 1(1), 80-106.

Liu, H., Lang, B., Liu, M., & Yan, H. (2019). CNN and RNN based payload classification methods for attack detection. *Knowledge-Based Systems*, 163, 332-341.

Mandiant. (2024). M-Trends 2024: Cyber Security Insights. <https://www.mandiant.com/m-trends>

McMahan, H. B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. A. (2017). Communication-efficient learning of deep networks from decentralized data. *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, 1273-1282.

Milajerdi, S. M., Gjomemo, R., Eshete, B., Sekar, R., & Venkatakrishnan, V. N. (2019). HOLMES: Real-time APT detection through correlation of suspicious information flows. *2019 IEEE Symposium on Security and Privacy (SP)*, 1137-1152.

MISP Project. (2024). MISP - Open Source Threat Intelligence Platform. <https://www.misp-project.org/>

MITRE Corporation. (2023). MITRE ATT&CK Framework. <https://attack.mitre.org/>

Nguyen, D. C., Ding, M., Pathirana, P. N., & Seneviratne, A. (2021). Federated learning for Internet of Things: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 23(3), 1622-1658.

NIST. (2018). Framework for Improving Critical Infrastructure Cybersecurity, Version 1.1. National Institute of Standards and Technology.

Noel, S., Jajodia, S., O'Berry, L., & Jacobs, M. (2004). Efficient minimum-cost network hardening via exploit dependency graphs. *Proceedings of the 19th Annual Computer Security Applications Conference*, 86-95.

Security Onion Solutions. (2024). Security Onion Documentation. <https://securityonionsolutions.com/>

Strom, B. E., Applebaum, A., Miller, D. P., Nickels, K. C., Pennington, A. G., & Thomas, C. B. (2018). MITRE ATT&CK: Design and philosophy. Technical Report, The MITRE Corporation.

Tanzania Data Protection Act. (2022). United Republic of Tanzania.

Vectra AI. (2023). Cognito Platform: AI-Driven Threat Detection. <https://www.vectra.ai/>

Wazuh Documentation. (2024). Wazuh - The Open Source Security Platform. <https://documentation.wazuh.com/>